3. Prediction with High Dimensional Predictors

Lieven Clement

Invalid Date

Table of contents

1 Introduction **3**
1.1 Prediction with High Dimensional Predictors . . . 3
1.2 Example: Toxicogenomics in early drug development . . . 3
1.2.1 Background . . . 3
1.2.2 Data . . . 4
1.2.3 Data exploration . . . 4
1.2.4 Prediction model . . . 8
1.3 Brain example . . . 9
1.3.1 Data exploration . . . 9
1.3.2 Models . . . 10
1.4 Overview . . . 14

2 Linear Regression for High Dimensional Data **15**
2.1 Linear Regression for multivariate data vs High Dimensional Data . . . 15
2.2 Can SVD help? . . . 16

3 Principal Component Regression **16**
3.0.1 Example model based on first 4 PCs . . . 17

4 Ridge Regression **18**
4.1 Penalty . . . 18
4.2 Graphical interpretation . . . 19
4.3 Solution . . . 19
4.3.1 Proof . . . 19
4.4 Link with SVD . . . 20
4.4.1 SVD of X when p > n . . . 20
4.4.2 SVD and inverse . . . 21

4.4.3 SVD of penalised matrix $\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I}$ . . . 21
4.4.4 Ridge estimator in terms of the SVD . . . 22
4.4.5 Numerical check . . . 22
4.5 Properties . . . 23
4.6 Toxicogenomics example . . . 25

5 Lasso Regression **26**
5.1 Graphical interpretation of Lasso vs ridge . . . 27
5.2 Toxicogenomics example . . . 28

6 Splines and the connection to ridge regression. **29**
6.1 Lidar dataset . . . 29
6.2 Basis expansion . . . 31
6.2.1 Truncated line basis . . . 31

7 Evaluation of Prediction Models **34**
7.1 Test or Generalisation Error . . . 35
7.2 Conditional test error . . . 35
7.3 Insample Error . . . 36
7.4 Estimation of the insample error . . . 36
7.4.1 Training error . . . 36
7.4.2 Proof . . . 37
7.5 Expected test error . . . 38
7.5.1 Estimating the Expected test error . . . 39
7.5.2 Bias Variance trade-off . . . 41
7.5.3 In practice . . . 41
7.5.4 Toxicogenomics example . . . 42
7.5.5 Lidar Example: splines . . . 48
7.6 More general error definitions . . . 49
7.6.1 Akaike's Information Criterion (AIC) . . . 50
7.6.2 Bayesian Information Criterion (BIC) . . . 51
7.7 Training and test sets . . . 52

8 Logistic Regression Analysis for High Dimensional Data **53**
8.1 Breast Cancer Example . . . 53
8.2 Data . . . 53
8.3 Logistic regression models . . . 54
8.4 Penalized maximum likelihood . . . 55
8.5 Model evaluation . . . 56
8.5.1 Sensitivity of a model π with threshold c . . . 56
8.5.2 Specificity of a model π with threshold c . . . 56
8.5.3 Misclassification error of a model π with threshold c . . . 57
8.5.4 ROC curve of a model π . . . 57

8.5.5 Area under the curve (AUC) of a model π . . . 57
8.6 Breast cancer example . . . 58
8.6.1 Data . . . 58
8.6.2 Training and test dataset . . . 59
8.6.3 Model fitting based on training data . . . 60
8.7 The Elastic Net . . . 65
8.7.1 Breast cancer example . . . 66

Acknowledgement **70**

Session info **70**

1 Introduction

1.1 Prediction with High Dimensional Predictors

General setting:

- Aim: build a **prediction model** that gives a prediction of an outcome for a given set of predictors.
- We use X to refer to the predictors and Y to refer to the outcome.
- A **training data set** is available, say $(\mathbf{X}, \mathbf{Y})$. It contains n observations on outcomes and on p predictors.
- Using the training data, a prediction model is build, say $\hat{m}(\mathbf{X})$. This typically involves **model building (feature selection)** and parameter estimation.
- During the model building, potential **models need to be evaluated** in terms of their prediction quality.

1.2 Example: Toxicogenomics in early drug development

1.2.1 Background

- Effect of compound on gene expression.
- Insight in action and toxicity of drug in early phase
- Determine activity with bio-assay: e.g. binding affinity of compound to cell wall receptor (target, IC50).
- Early phase: 20 to 50 compounds

- Based on in vitro results one aims to get insight in how to build better compound (higher on-target activity less toxicity.
- Small variations in molecular structure lead to variations in BA and gene expression.
- Aim: Build model to predict bio-activity based on gene expression in liver cell line.

1.2.2 Data

- 30 chemical compounds have been screened for toxicity
- Bioassay data on toxicity screening
- Gene expressions in a liver cell line are profiled for each compound (4000 genes)

```
toxData <- read_csv(
  "https://raw.githubusercontent.com/statOmics/HDA2020/data/toxDataCentered.csv",
  col_types = cols()
)
svdX <- svd(toxData[,-1])
```

Data is already centered:

```
toxData |>
  colMeans() |>
  range()
```

```
#> [1] -2.405483e-17  5.921189e-17
```

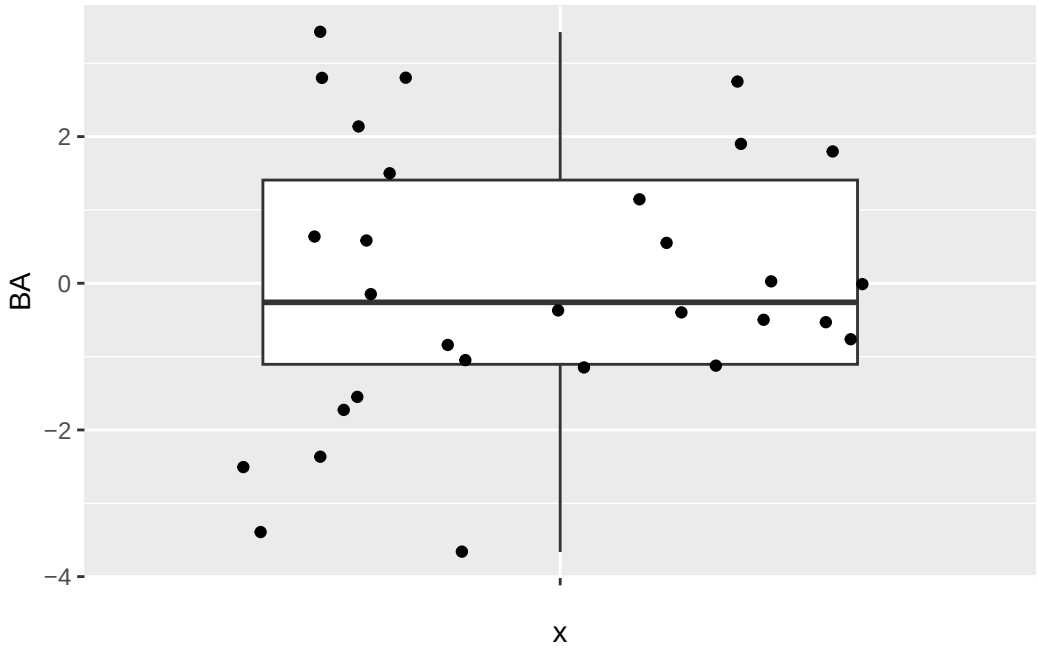
```
 toxData |>
  names() |>
  head()
```

```
#> [1] "BA" "X1" "X2" "X3" "X4" "X5"
```

- First column contains data on Bioassay.
- The higher the score on Bioassay the more toxic the compound
- Other columns contain data on gene expression X1, … , X4000

1.2.3 Data exploration

```r
toxData |>
  ggplot(aes(x="",y=BA)) +
  geom_boxplot(outlier.shape=NA) +
  geom_point(position="jitter")
```

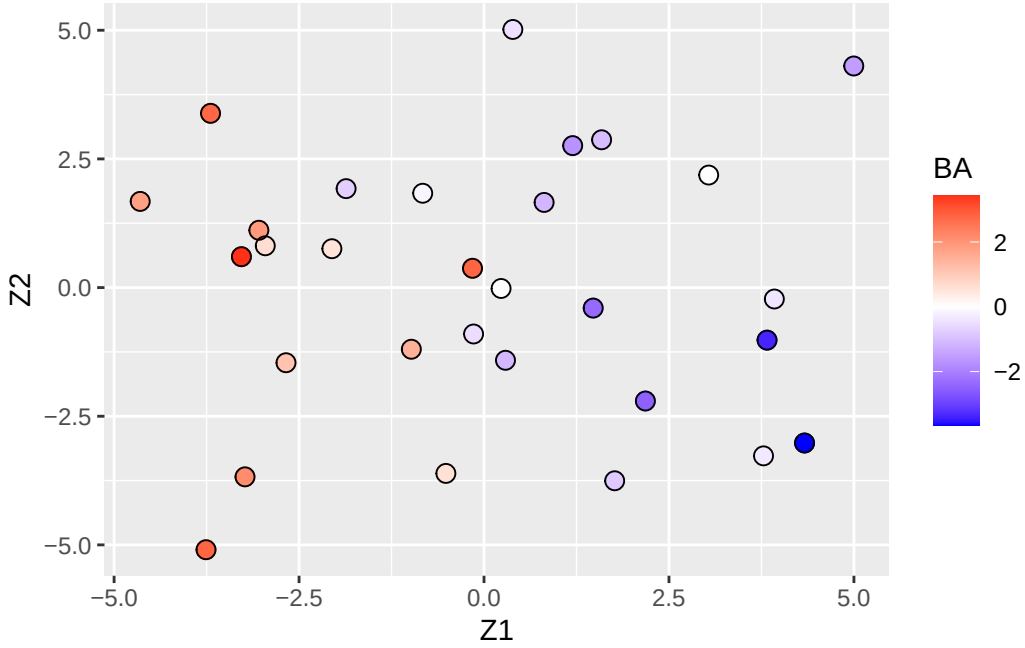



```r
svdX <- toxData[,-1] |>
  svd()

k <- 2
Vk <- svdX$v[,1:k]
Uk <- svdX$u[,1:k]
Dk <- diag(svdX$d[1:k])
Zk <- Uk%*%Dk
colnames(Zk) <- paste0("Z",1:k)
colnames(Vk) <- paste0("V",1:k)

Zk |>
  as.data.frame() |>
  mutate(BA = toxData |> pull(BA)) |>
  ggplot(aes(x= Z1, y = Z2, color = BA)) +
  geom_point(size = 3) +
```

```
  scale_colour_gradient2(low = "blue",mid="white",high="red") +
  geom_point(size = 3, pch = 21, color = "black")
```



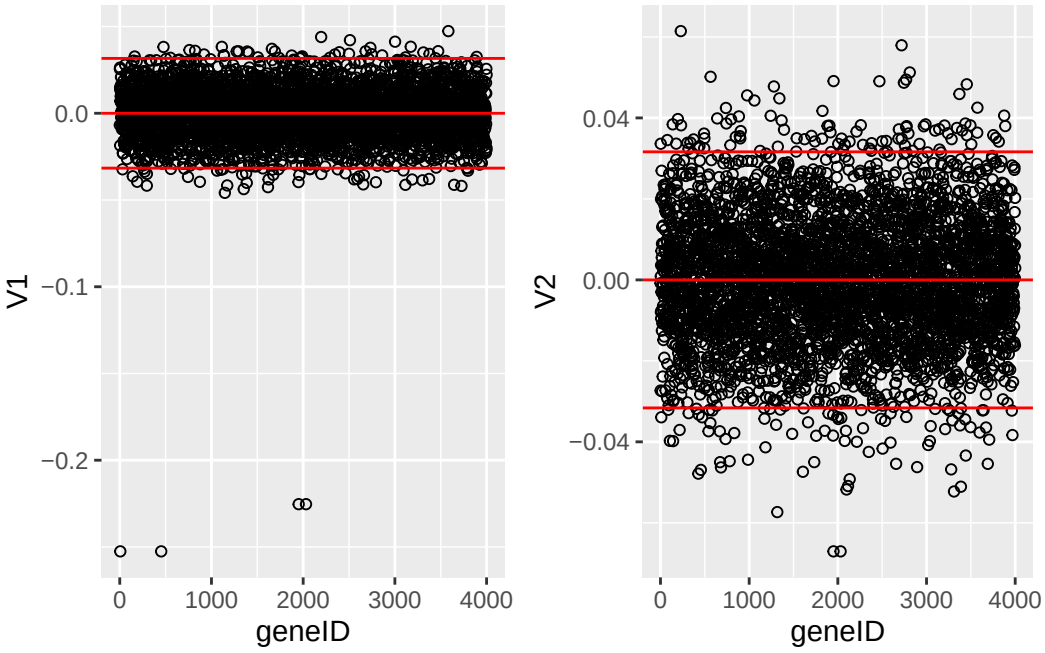
- Scores on the first two principal components (or MDS plot).
- Each point corresponds to a compound.
- Color code refers to the toxicity score (higher score more toxic).
- Clear separation between compounds according to toxicity.

- Next logic step in a PCA is to interpret the principal components.
- We thus have to assess the loadings.
- We can add a vector for each gene to get a biplot, but this would require plotting 4000 vectors, which would render the plot unreadable.

Alternative graph to look at the many loadings of the first two PCs.

```
grid.arrange(
  Vk |>
    as.data.frame() |>
    mutate(geneID = 1:nrow(Vk)) |>
    ggplot(aes(x = geneID, y = V1)) +
```

```
  geom_point(pch=21) +
  geom_hline(yintercept = c(-2,0,2)*sd(Vk[,1]), col = "red") ,
Vk |>
  as.data.frame() |>
  mutate(geneID = 1:nrow(Vk)) |>
  ggplot(aes(x = geneID, y = V2)) +
  geom_point(pch=21) +
  geom_hline(yintercept = c(-2,0,2)*sd(Vk[,2]), col = "red"),
ncol=2)
```



- It is almost impossible to interpret the PCs because there are 4000 genes contributing to each PC.
- In an attempt to find the most important genes (in the sense that they drive the interpretation of the PCs), the plots show horizontal reference lines: the average of the loadings, and the average $\pm$ twice the standard deviation of the loadings. In between the lines we expects about 95% of the loadings (if they were normally distributed).
- The points outside the band come from the genes that have rather large loadings (in absolute value) and hence are important for the interpretation of the PCs.
- Note, that particularly for the first PC, only a few genes show a markedly large loadings that are negative. This means that an upregulation of these genes will lead to low scores on PC1.

- These genes will very likely play an important role in the toxicity mechanism.
- Indeed, low scores on PC1 are in the direction of more toxicity.
- In the next chapter we will introduce a method to obtain sparse PCs.

1.2.4 Prediction model

```
m1 <- lm(BA ~ -1 + ., toxData)

m1 |>
  coef() |>
  head(40)
```

```
#>          X1          X2          X3          X4          X5          X6
#>  -7.4569940   0.3571348  11.2492315  10.8354021 -13.7433891   5.6833874
#>          X7          X8          X9         X10         X11         X12
#>  65.5387777   4.3404555   7.9103924  37.0296057 -54.8368698 -55.5547845
#>         X13         X14         X15         X16         X17         X18
#>   5.7924667  23.1428002  -6.9610365 -28.5250571 -22.5509025 -97.9623731
#>         X19         X20         X21         X22         X23         X24
#> -30.4171782 -32.6991673 -14.2808834 -16.1431266 -22.7498681  73.1635178
#>         X25         X26         X27         X28         X29         X30
#>  -5.7065827  37.4745379 -20.1999102  14.9906821  99.6080955          NA
#>         X31         X32         X33         X34         X35         X36
#>          NA          NA          NA          NA          NA          NA
#>         X37         X38         X39         X40
#>          NA          NA          NA          NA
```

```
m1 |>
  coef() |>
  is.na() |>
  sum()
```

```
#> [1] 3971
```

```
summary(m1)$r.squared
```

```
#> [1] 1
```

Problem??

1.3 Brain example

- Courtesy to Solomon Kurz. Statistical rethinking with brms, ggplot2, and the tidyverse version 1.2.0.

https://bookdown.org/content/3890/ https://github.com/ASKurz/Statistical_Rethinking_with_brms_ggplot

- Data with brain size and body size for seven species

```
brain <-
tibble(species = c("afarensis", "africanus", "habilis", "boisei", "rudolfensis", "ergaster",
       brain   = c(438, 452, 612, 521, 752, 871, 1350),
       mass    = c(37.0, 35.5, 34.5, 41.5, 55.5, 61.0, 53.5))
```

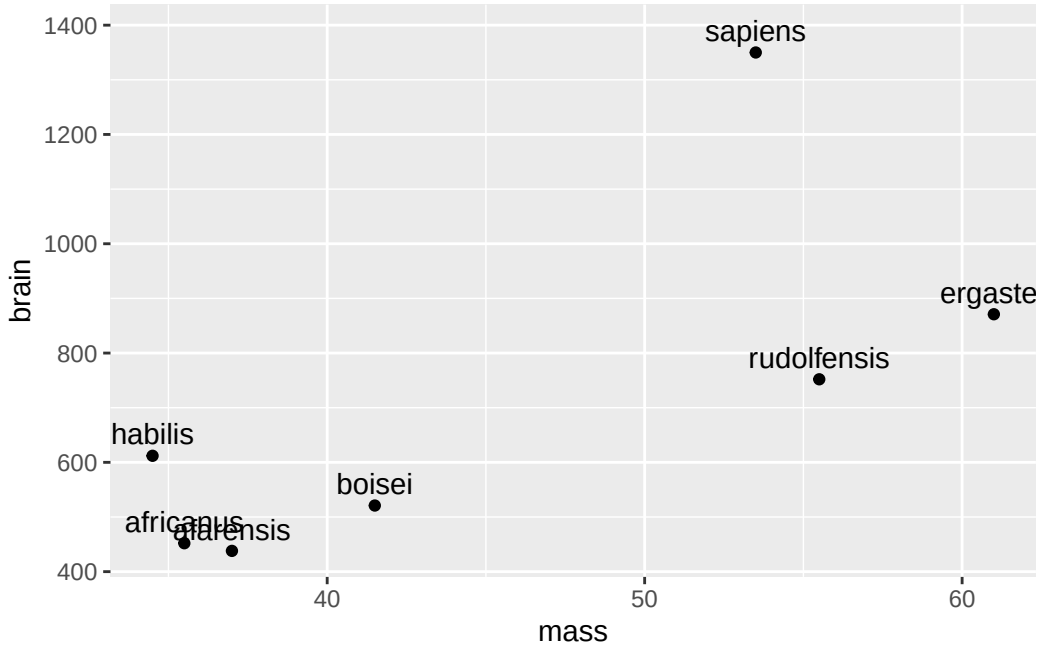
1.3.1 Data exploration

```
brain
```

```
#> # A tibble: 7 x 3
#>   species     brain  mass
#>   <chr>       <dbl> <dbl>
#> 1 afarensis     438  37
#> 2 africanus     452  35.5
#> 3 habilis       612  34.5
#> 4 boisei        521  41.5
#> 5 rudolfensis   752  55.5
#> 6 ergaster      871  61
#> 7 sapiens      1350  53.5
```

```
p <- brain |>
  ggplot(aes(x =  mass, y = brain, label = species)) +
  geom_point()

p + geom_text(nudge_y = 40)
```



1.3.2 Models

Six models range in complexity from the simple univariate model

$$\begin{aligned} \text{brain}_i &\sim \text{Normal}(\mu_i, \sigma) \\ \mu_i &= \beta_0 + \beta_1 \text{mass}_i, \end{aligned}$$

to the dizzying sixth-degree polynomial model

$$\begin{aligned} \text{brain}_i &\sim \text{Normal}(\mu_i, \sigma) \\ \mu_i &= \beta_0 + \beta_1 \text{mass}_i + \beta_2 \text{mass}_i^2 + \beta_3 \text{mass}_i^3 + \beta_4 \text{mass}_i^4 + \beta_5 \text{mass}_i^5 + \beta_6 \text{mass}_i^6. \end{aligned}$$

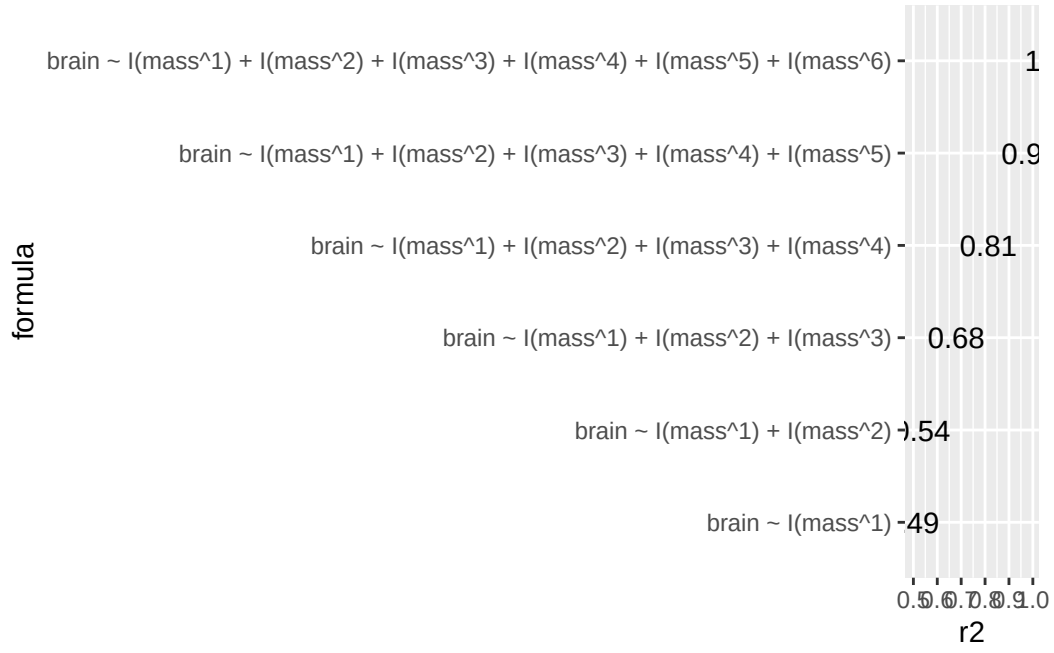
```
formulas <- sapply(1:6, function(i)
  return(
     paste0("I(mass^",1:i,")") |> paste(collapse=" + ")
    )
)

formulas <- sapply(
```

```
  paste0("brain ~ ", formulas),
  as.formula)

models <- lapply(formulas, lm , data = brain)
```

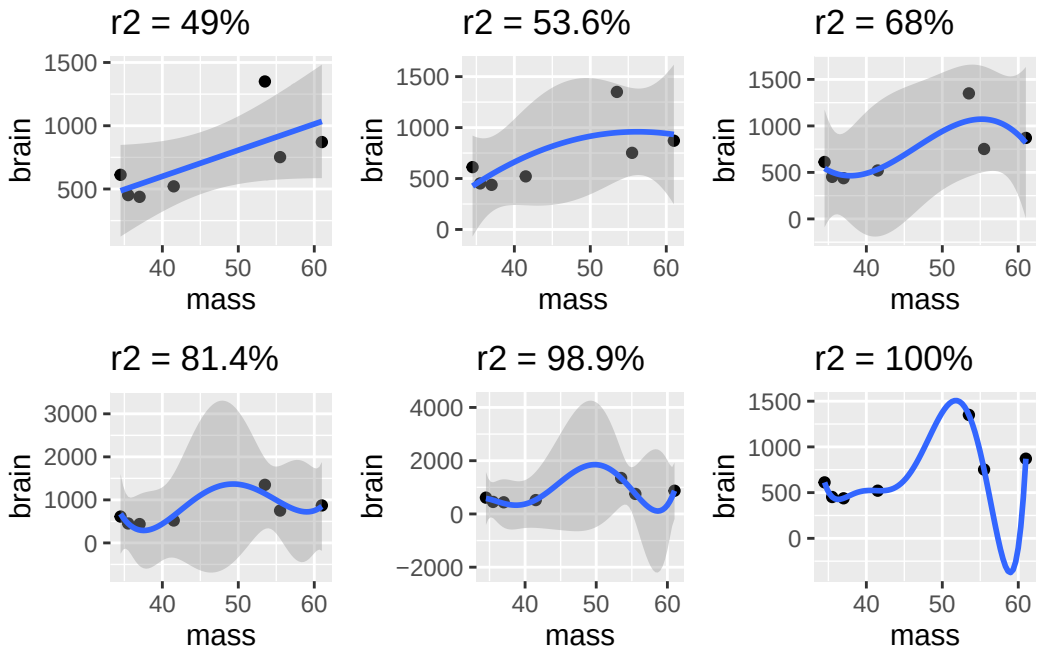
```
data.frame(
  formula=formulas |>
    as.character(),
  r2 = sapply(
    models,
    function(mod) summary(mod)$r.squared)
  )  |>
  ggplot(
    aes(x = r2,
      y = formula,
      label = r2 |>
        round(2) |>
        as.character())
  ) +
  geom_text()
```



We plot the fit for each model individually and them arrange them together in one plot.

```r
plots <- lapply(1:6, function(i)
{
  p +
  geom_smooth(method = "lm", formula = y ~ poly(x,i)) +
  ggtitle(
    paste0(
      "r2 = ",
      round(summary(models[[i]])$r.squared*100,1),
      "%")
    )
})

do.call("grid.arrange",c(plots, ncol = 3))
```

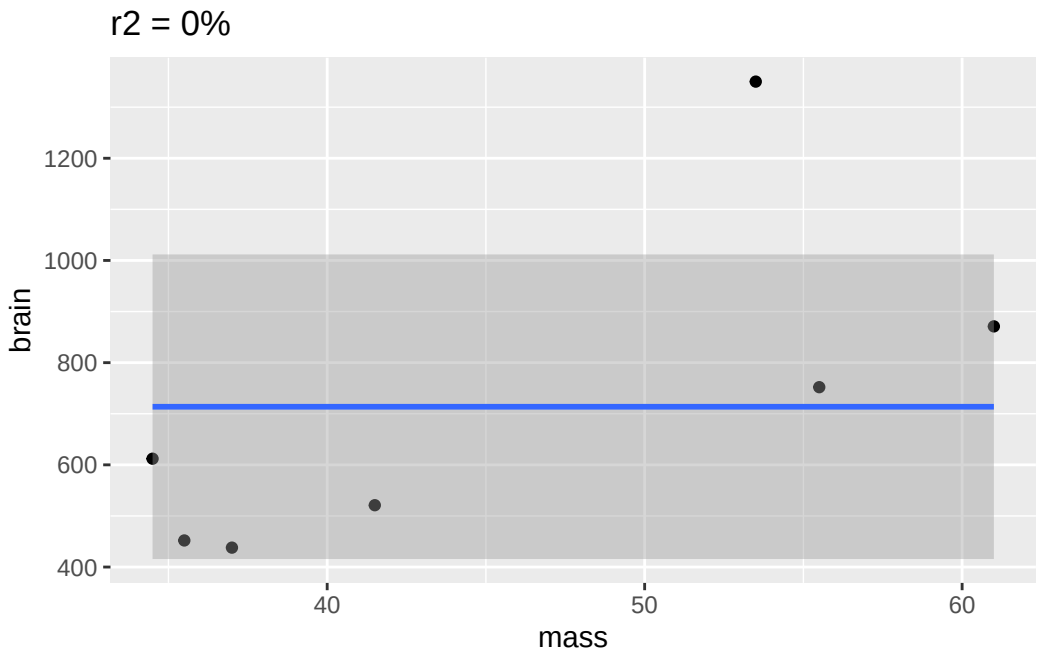


- We clearly see that increasing the model complexity always produces a fit with a smaller SSE.
- The problem of overfitting is very obvious. The more complex polynomial models will not generalise well for prediction!
- We even have a model that fits the data perfectly, but that will make very absurd predictions!
- Too few parameters hurts, too. Fit the underfit intercept-only model.

```r
m0 <- lm(brain ~ 1, brain)
summary(m0)
```

```
#>
#> Call:
#> lm(formula = brain ~ 1, data = brain)
#>
#> Residuals:
#>     Min      1Q  Median      3Q     Max
#> -275.71 -227.21 -101.71   97.79  636.29
#>
#> Coefficients:
#>             Estimate Std. Error t value Pr(>|t|)
#> (Intercept)    713.7      121.8    5.86  0.00109 **
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 322.2 on 6 degrees of freedom
```

```r
p +
  stat_smooth(method = "lm", formula = y ~ 1) +
  ggtitle(
    paste0(
      "r2 = ",
      round(summary(m0)$r.squared*100,1),
      "%")
    )
```



The underfit model did not learn anything about the relation between mass and brain. It would also do a very poor job for predicting new data.

1.4 Overview

We will make a distinction between continuous and discrete outcomes. In this course we focus on

- Linear regression models for continuous outcomes
  - Penalised regression: Lasso and ridge
  - Principal component regression (PCR)
- Logistic regression models for binary outcomes
  - Penalised regression: Lasso and ridge

For all types of model, we will discuss feature selection methods.

2 Linear Regression for High Dimensional Data

Consider linear regression model (for double centered data)

$$Y_i = \beta_1 X_{i1} + \beta_2 X_{i2} + \cdots + \beta_p X_{ip} + \epsilon_i,$$

with $\mathrm{E}[\epsilon \mid \mathbf{X}] = 0$ and $\mathrm{var}[\epsilon \mid \mathbf{X}] = \sigma^2$.

In matrix notation the model becomes

$$\mathbf{Y} = \mathbf{X} \; + \; .$$

The least squares estimator of is given by

$$\hat{} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y},$$

and the variance of $\hat{}$ equals

$$\mathrm{var}[\hat{}] = (\mathbf{X}^T\mathbf{X})^{-1}\sigma^2.$$

$\longrightarrow$ the $p \times p$ matrix $(\mathbf{X}^T\mathbf{X})^{-1}$ is crucial

Note, that

- with double centered data it is meant that both the responses are centered (mean of $\mathbf{Y}$ is zero) and that all predictors are centered (columns of $\mathbf{X}$ have zero mean). With double centered data the intercept in a linear regression model is always exactly equal to zero and hence the intercept must not be included in the model.
- we do not assume that the residuals are normally distributed. For prediction purposes this is often not required (normality is particularly important for statistical inference in small samples).

2.1 Linear Regression for multivariate data vs High Dimensional Data

- $\mathbf{X}^T\mathbf{X}$ and $(\mathbf{X}^T\mathbf{X})^{-1}$ are $p \times p$ matrices
- $\mathbf{X}^T\mathbf{X}$ can only be inverted if it has rank p
- Rank of a matrix of form $\mathbf{X}^T\mathbf{X}$, with $\mathbf{X}$ and $n \times p$ matrix, can never be larger than $\min(n, p)$.
- in most regression problems $n > p$ and rank of $(\mathbf{X}^T\mathbf{X})$ equals p
- in high dimensional regression problems $p >>> n$ and rank of $(\mathbf{X}^T\mathbf{X})$ equals $n < p$
- in the toxicogenomics example $n = 30 < p = 4000$ and the columns of $\mathbf{X}$ are centered, so $\mathrm{rank}(\mathbf{X}^T\mathbf{X}) \leq n - 1 = 29$. $\longrightarrow$ $(\mathbf{X}^T\mathbf{X})^{-1}$ does not exist, and neither does $\hat{\beta}$.

2.2 Can SVD help?

- Since the columns of $\mathbf{X}$ are centered, $\mathbf{X}^T\mathbf{X} \propto \text{var}[\mathbf{X}]$.
- Centering the columns of $\mathbf{X}$ reduces the rank by one: the n centered rows sum to zero, so they span at most an $(n-1)$-dimensional subspace. In the toxicogenomics example $\text{rank}(\mathbf{X}) = \text{rank}(\mathbf{X}^T\mathbf{X}) = r = n - 1 = 29$, and the 30th singular value is zero.
- Hence, the PCA will give $r = 29$ components, each being a linear combination of $p = 4000$ variables. These 29 PCs contain all information present in the original $\mathbf{X}$ data.
- The SVD of $\mathbf{X}$ is given by

$$\mathbf{X} = \sum_{k=1}^{r} \delta_k \mathbf{u}_k \mathbf{v}_k^T = \mathbf{U}\Delta\mathbf{V}^T = \mathbf{Z}\mathbf{V}^T,$$

  with $\mathbf{U}$ an $n \times r$ matrix, Δ an $r \times r$ diagonal matrix, $\mathbf{V}$ a $p \times r$ matrix, and $\mathbf{Z} = \mathbf{U}\Delta$ the $n \times r$ matrix with the scores on the r PCs.
- Still problematic: if we use all $r = n - 1$ PCs as regressors, the model fits the data perfectly with zero residual degrees of freedom (centering $\mathbf{Y}$ already used one degree of freedom).

3 Principal Component Regression

A principal component regression (PCR) consists of

1. transforming $p = 4000$ dimensional X-variable to the $r = n - 1 = 29$ dimensional Z-variable (PC scores). The r PCs are mutually uncorrelated.
2. using the r PC-variables as regressors in a linear regression model
3. performing feature selection to select the most important regressors (PC).

Feature selection is key, because we don't want to have as many regressors as there are observations in the data. This would result in zero residual degrees of freedom. (see later)

To keep the exposition general so that we allow for a feature selection to have taken place, I use the notation $\mathbf{U}_S$ to denote a matrix with left-singular column vectors $\mathbf{u}_k$, with $k \in \mathcal{S}$ ($\mathcal{S}$ an index set referring to the PCs to be included in the regression model).

For example, suppose that a feature selection method has resulted in the selection of PCs 1, 3 and 12 for inclusion in the prediction model, then $\mathcal{S} = \{1, 3, 12\}$ and

$$\mathbf{U}_S = \begin{pmatrix} \mathbf{u}_1 & \mathbf{u}_3 & \mathbf{u}_{12} \end{pmatrix}.$$

3.0.1 Example model based on first 4 PCs

```
k <- 29 # rank of the centered X: the 30th singular value is zero
Uk <- svdX$u[,1:k]
Dk <- diag(svdX$d[1:k])
Zk <- Uk%*%Dk
Y <- toxData |>
  pull(BA)

m4 <- lm(Y~Zk[,1:4])
summary(m4)
```

```
#>
#> Call:
#> lm(formula = Y ~ Zk[, 1:4])
#>
#> Residuals:
#>     Min      1Q  Median      3Q     Max
#> -2.1438 -0.7033 -0.1222  0.7255  2.2997
#>
#> Coefficients:
#>               Estimate Std. Error t value Pr(>|t|)
#> (Intercept)  5.158e-16  2.081e-01   0.000   1.0000
#> Zk[, 1:4]1  -5.275e-01  7.725e-02  -6.828 3.72e-07 ***
#> Zk[, 1:4]2  -1.231e-02  8.262e-02  -0.149   0.8828
#> Zk[, 1:4]3  -1.759e-01  8.384e-02  -2.098   0.0461 *
#> Zk[, 1:4]4  -3.491e-02  8.396e-02  -0.416   0.6811
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 1.14 on 25 degrees of freedom
#> Multiple R-squared:  0.672,  Adjusted R-squared:  0.6195
#> F-statistic:  12.8 on 4 and 25 DF,  p-value: 8.352e-06
```

Note:

- the intercept is estimated as zero. (Why?) The model could have been fitted as

```
m4 <- lm(Y~-1+Zk[,1:4])
```

- the PC-predictors are uncorrelated (by construction)
- first PC-predictors are not necessarily the most important predictors
- p-values are not very meaningful when prediction is the objective

Methods for feature selection will be discussed later.

4 Ridge Regression

4.1 Penalty

The ridge parameter estimator is defined as the parameter that minimises the **penalised least squares criterion**

$$\text{SSE}_{\text{pen}} = \|\mathbf{Y} - \mathbf{X}\|_2^2 + \lambda\|\beta\|_2^2$$

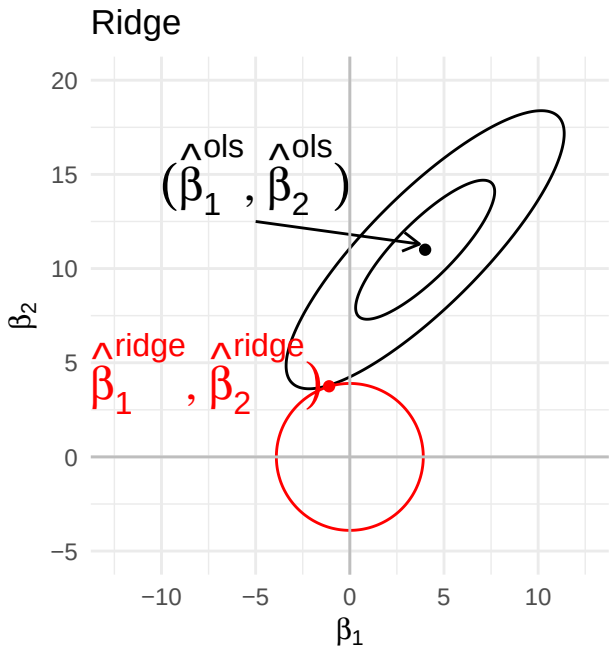
- $\|\beta\|_2^2 = \sum_{j=1}^{p} \beta_j^2$ is the L_2 **penalty term**
- $\lambda > 0$ is the penalty parameter (to be chosen by the user).

Note, that that is equivalent to minimizing

$$\|\mathbf{Y} - \mathbf{X}\|_2^2 \text{ subject to } \|\beta\|_2^2 \leq s$$

Note, that s has a one-to-one correspondence with λ

4.2 Graphical interpretation



4.3 Solution

The solution is given by

$$\hat{\beta} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{Y}.$$

It can be shown that $(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})$ is always of rank p if $\lambda > 0$.

Hence, $(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})$ is invertible and $\hat{\beta}$ exists even if $p >>> n$.

We also find

$$\text{var}[\hat{\ }] = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{X}(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\sigma^2$$

4.3.1 Proof

The criterion to be minimised is

$$\text{SSE}_{\text{pen}} = \|\mathbf{Y} - \mathbf{X}\|_2^2 + \lambda\|\beta\|_2^2.$$

First we re-express SSE in matrix notation:

$$\text{SSE}_{\text{pen}} = (\mathbf{Y} - \mathbf{X})^T(\mathbf{Y} - \mathbf{X}) + \lambda\beta^T\beta.$$

The partial derivative w.r.t. β is

$$\frac{\partial}{\partial \beta} \text{SSE}_{\text{pen}} = -2\mathbf{X}^T(\mathbf{Y} - \mathbf{X}\,) + 2\lambda\beta.$$

Solving $\frac{\partial}{\partial \beta} \text{SSE}_{\text{pen}} = 0$ gives

$$\widehat{\beta} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{Y}.$$

(assumption: $(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})$ is of rank p. This is always true if $\lambda > 0$)

4.4 Link with SVD

4.4.1 SVD of X when p > n

Write the SVD of $\mathbf{X}$ ($n \times p$, rank r) as

$$\mathbf{X} = \sum_{k=1}^{r} \delta_k \mathbf{u}_k \mathbf{v}_k^T = \mathbf{U}\Delta\mathbf{V}^T,$$

with

- $\mathbf{U}$ an $n \times r$ matrix with orthonormal columns, Δ an $r \times r$ diagonal matrix with $\delta_1 \geq \ldots \geq \delta_r > 0$,
- $\mathbf{V}$ a $p \times r$ matrix with orthonormal columns: $\mathbf{V}^T\mathbf{V} = \mathbf{I}_r$, but $\mathbf{V}\mathbf{V}^T \neq \mathbf{I}_p$ when $r < p$.

This is the "thin" SVD.

Note, that `svd()` in R returns $\min(n, p)$ singular vectors: the columns with $\delta_k \approx 0$ have to be removed to obtain the rank-r version.

Geometrically:

- we only have n data points $\mathbf{x}_1, \ldots, \mathbf{x}_n$ in a p-dimensional space.
- n points span at most an n-dimensional subspace, just like 2 points (and the origin) span a plane in $\mathbb{R}^3$.
- Hence, $r \leq \min(n, p)$, and when $p > n$ we have $r \leq n < p$.
- The columns $\mathbf{v}_1, \ldots, \mathbf{v}_r$ are an orthonormal basis of the subspace spanned by the data: every data point is a linear combination of them, $\mathbf{x}_i^T = \sum_{k=1}^{r} u_{ik}\delta_k\mathbf{v}_k^T$.
- $\mathbf{V}\mathbf{V}^T$ projects onto this subspace; these are the directions in which the data contain information.
- The remaining $p - r$ directions are perpendicular to all data points.
- We collect them in $p - r$ orthonormal columns $\mathbf{V}_\perp$.

- All data points have coordinate zero in these directions ($\mathbf{X}\mathbf{V}_\perp = \mathbf{0}$): the data contain no information there. Then $\mathbf{V}_{\text{full}} = [\mathbf{V}\ \mathbf{V}_\perp]$ is a $p \times p$ orthogonal matrix and

$$\mathbf{V}\mathbf{V}^T + \mathbf{V}_\perp\mathbf{V}_\perp^T = \mathbf{I}_p.$$

- Note, that centering the columns of $\mathbf{X}$ reduces the rank by one: in the double centered setting of this chapter with $p > n$, $r = n - 1$.

4.4.2 SVD and inverse

With the SVD of $\mathbf{X}$ we write

$$\mathbf{X}^T\mathbf{X} = \mathbf{V}\Delta^2\mathbf{V}^T = \mathbf{V}_{\text{full}} \begin{bmatrix} \Delta^2 & \mathbf{0} \\ \mathbf{0} & \mathbf{0}_{(p-r)\times(p-r)} \end{bmatrix} \mathbf{V}_{\text{full}}^T.$$

- $\mathbf{X}^T\mathbf{X}$ is a $p \times p$ matrix with $p - r$ zero eigenvalues.
- Hence, when $r < p$ it is singular and $(\mathbf{X}^T\mathbf{X})^{-1}$ does not exist.
- The matrix $\mathbf{V}\Delta^{-2}\mathbf{V}^T$ is not the inverse.
- It only inverts $\mathbf{X}^T\mathbf{X}$ on the span of $\mathbf{V}$:

$$(\mathbf{X}^T\mathbf{X})\,\mathbf{V}\Delta^{-2}\mathbf{V}^T = \mathbf{V}\mathbf{V}^T \neq \mathbf{I}_p.$$

- It is the Moore-Penrose pseudo-inverse $(\mathbf{X}^T\mathbf{X})^+$.
- Note, however, that when $n > p$ and $\mathbf{X}$ has full rank $r = p$, $\mathbf{V}$ is a $p \times p$ orthogonal matrix, and $(\mathbf{X}^T\mathbf{X})^{-1} = \mathbf{V}\Delta^{-2}\mathbf{V}^T$ does hold.

4.4.3 SVD of penalised matrix $\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I}$

Adding λ to the diagonal does not change the eigenvectors, and increases all eigenvalues by λ:

$$\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I} = \mathbf{V}_{\text{full}} \begin{bmatrix} \Delta^2 + \lambda\mathbf{I}_r & \mathbf{0} \\ \mathbf{0} & \lambda\mathbf{I}_{p-r} \end{bmatrix} \mathbf{V}_{\text{full}}^T = \mathbf{V}(\Delta^2 + \lambda\mathbf{I})\mathbf{V}^T + \lambda\mathbf{V}_\perp\mathbf{V}_\perp^T.$$

Note, that

- with the thin $\mathbf{V}$ alone,

$$\mathbf{V}(\Delta^2 + \lambda\mathbf{I})\mathbf{V}^T = \mathbf{X}^T\mathbf{X} + \lambda\mathbf{V}\mathbf{V}^T \neq \mathbf{X}^T\mathbf{X} + \lambda\mathbf{I}:$$

  the penalty in the $\mathbf{V}_\perp$ directions is missing.

- the $p-r$ zero eigenvalues become λ, so all eigenvalues are strictly positive and the matrix is invertible for every $\lambda > 0$.
- Because $\mathbf{V}_{\text{full}}$ is orthogonal ($\mathbf{V}_{\text{full}}^{-1} = \mathbf{V}_{\text{full}}^T$), the inverse is obtained by inverting the diagonal matrix in the middle:

$$\begin{aligned}(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1} &= \mathbf{V}_{\text{full}} \begin{bmatrix} \Delta^2 + \lambda\mathbf{I}_r & \mathbf{0} \\ \mathbf{0} & \lambda\mathbf{I}_{p-r} \end{bmatrix}^{-1} \mathbf{V}_{\text{full}}^T \\ &= \mathbf{V}(\Delta^2 + \lambda\mathbf{I})^{-1}\mathbf{V}^T + \frac{1}{\lambda}\mathbf{V}_\perp\mathbf{V}_\perp^T.\end{aligned}$$

$\longrightarrow$ Interestingly, when $n > p$ and $\mathbf{X}$ is full rank $r = p$, $\mathbf{V}$ is a $p \times p$ orthogonal matrix.

$\longrightarrow$ Then $\mathbf{V}\mathbf{V}^T = \mathbf{I}_p$, there are no $\mathbf{V}_\perp$ directions, and

$$\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I} = \mathbf{V}(\Delta^2 + \lambda\mathbf{I})\mathbf{V}^T, \qquad (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1} = \mathbf{V}(\Delta^2 + \lambda\mathbf{I})^{-1}\mathbf{V}^T.$$

4.4.4 Ridge estimator in terms of the SVD

- Fortunately, the $\mathbf{V}_\perp$ term drops out of the ridge estimator.
- Indeed $\mathbf{X}^T\mathbf{Y} = \mathbf{V}\Delta\mathbf{U}^T\mathbf{Y}$ lies in the span of $\mathbf{V}$, and $\mathbf{V}_\perp^T\mathbf{V} = \mathbf{0}$.
- Hence,

$$\hat{\beta} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{Y} = \mathbf{V}(\Delta^2 + \lambda\mathbf{I})^{-1}\Delta\mathbf{U}^T\mathbf{Y} = \sum_{k=1}^{r} \frac{\delta_k}{\delta_k^2 + \lambda}\, \mathbf{v}_k\, \mathbf{u}_k^T\mathbf{Y}.$$

- So the ridge estimator can be computed from the thin SVD alone, and it always lies in the span of $\mathbf{V}$.
- And, the ridge estimator is the same for both $p \geq n$ and $p < n$.

4.4.5 Numerical check

```r
# local() keeps the simulated objects out of the workspace of this chapter
local({
  set.seed(1)
  n <- 10; p <- 50; lambda <- 2
  X <- scale(matrix(rnorm(n * p), n, p), scale = FALSE)
  Y <- rnorm(n); Y <- Y - mean(Y)
```

```r
  # svd() returns min(n, p) = n singular vectors
  sv <- svd(X)
  # last singular value is 0 due to centring
  print(round(sv$d, 10))
  r <- n - 1 # rank after centring
  d <- sv$d[1:r]
  # thin SVD: V is p x r
  V <- sv$v[, 1:r]; U <- sv$u[, 1:r]
  # remaining p - r directions
  Vperp <- svd(X, nv = p)$v[, (r + 1):p]

  A <- crossprod(X) + lambda * diag(p)
  B <- V %*% diag(d^2 + lambda) %*% t(V)

  # thin V only: not equal
  print(max(abs(B - A)))

  # thin V plus Vperp term: equal
  print(max(abs(B + lambda * tcrossprod(Vperp) - A)))

  # ridge estimator: thin SVD is enough
  beta_direct <- solve(A, crossprod(X, Y))
  beta_svd    <- V %*% (d / (d^2 + lambda) * crossprod(U, Y))
  print(max(abs(beta_direct - beta_svd)))
})
```

```
#>  [1] 9.890956 9.469785 8.240476 7.750660 7.074384 5.844626 5.028405 4.998850
#>  [9] 4.557865 0.000000
#> [1] 1.888063
#> [1] 4.618528e-14
#> [1] 6.661338e-16
```

4.5 Properties

- The ridge estimator is biased! The β are shrunken towards zero!

$$\begin{aligned} \mathrm{E}[\hat{\beta}] &= (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathrm{E}[\mathbf{Y}] \\ &= (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{X}\beta \end{aligned}$$

- Note, that the shrinkage is larger in the direction of the smaller eigenvalues. As for $\hat{\beta}$, the $\mathbf{V}_\perp$ term drops out because $\mathbf{X}^T\mathbf{X} = \mathbf{V}\Delta^2\mathbf{V}^T$, so the thin SVD suffices:

$$
\begin{aligned}
\mathrm{E}[\widehat{\beta}] &= \mathbf{V}(\Delta^2 + \lambda\mathbf{I})^{-1}\Delta^2\mathbf{V}^T\beta \\
&= \mathbf{V}\begin{bmatrix} \frac{\delta_1^2}{\delta_1^2+\lambda} & \cdots & 0 \\ & \ddots & \\ 0 & \cdots & \frac{\delta_r^2}{\delta_r^2+\lambda} \end{bmatrix}\mathbf{V}^T\beta \\
&= \sum_{k=1}^{r} \frac{\delta_k^2}{\delta_k^2+\lambda}\,\mathbf{v}_k\mathbf{v}_k^T\beta
\end{aligned}
$$

Compare this with the decomposition of the true parameter vector in the same basis,

$$
\beta = \mathbf{V}_{\text{full}}\mathbf{V}_{\text{full}}^T\beta = \sum_{k=1}^{r}\mathbf{v}_k\mathbf{v}_k^T\beta + \mathbf{V}_\perp\mathbf{V}_\perp^T\beta.
$$

Ridge thus shrinks most in the directions where the data say least: by a factor $\delta_k^2/(\delta_k^2+\lambda)$ along $\mathbf{v}_k$, and completely along $\mathbf{V}_\perp$. The component $\mathbf{V}_\perp\mathbf{V}_\perp^T\beta$ is shrunken to zero: the data contain no information on it.

- The variance of the ridge estimator is

$$
\mathrm{var}\left[\widehat{\beta}\right] = (\mathbf{X}^T\mathbf{X}+\lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{X}(\mathbf{X}^T\mathbf{X}+\lambda\mathbf{I})^{-1}\sigma^2 = \mathbf{V}\,\Delta^2(\Delta^2+\lambda\mathbf{I})^{-2}\,\mathbf{V}^T\sigma^2,
$$

  with diagonal elements $\frac{\delta_k^2}{(\delta_k^2+\lambda)^2}$ that decrease with λ.

- Compare this with the variance of the OLS estimator, which only exists when $\mathbf{X}$ has full rank $r = p$ ($n > p$):

$$
\begin{aligned}
\mathrm{var}\left[\widehat{\beta}_{\text{OLS}}\right] &= (\mathbf{X}^T\mathbf{X})^{-1}\sigma^2 = \sigma^2\sum_{k=1}^{r}\frac{1}{\delta_k^2}\mathbf{v}_k\mathbf{v}_k^T, \\
\mathrm{var}\left[\widehat{\beta}\right] &= \sigma^2\sum_{k=1}^{r}\frac{\delta_k^2}{(\delta_k^2+\lambda)^2}\mathbf{v}_k\mathbf{v}_k^T.
\end{aligned}
$$

  - Along each $\mathbf{v}_k$ the variance is reduced by a factor $\left(\frac{\delta_k^2}{\delta_k^2+\lambda}\right)^2$: the square of the shrinkage factor of $\mathrm{E}[\widehat{\beta}]$. The same factor that introduces bias reduces the variance.
  - The OLS variance $1/\delta_k^2$ explodes for small singular values (near collinear predictors), while the ridge variance is bounded: $\frac{\delta_k^2}{(\delta_k^2+\lambda)^2} \le \frac{1}{4\lambda}$.[1]
  - When $p > n$, $(\mathbf{X}^T\mathbf{X})^{-1}$ and the OLS variance do not exist, while the ridge variance is always finite. For $\lambda \to 0$ it converges to $\sigma^2\sum_{k=1}^{r}\mathbf{v}_k\mathbf{v}_k^T/\delta_k^2$, the variance of the minimum-norm least squares solution, which becomes very large when some δ_k are small.

[1] This follows from $(\delta_k^2-\lambda)^2 \ge 0 \Leftrightarrow (\delta_k^2+\lambda)^2 \ge 4\delta_k^2\lambda$, with equality if and only if $\delta_k^2 = \lambda$.

- The variance of the prediction $\hat{Y}(\mathbf{x}) = \mathbf{x}^T\hat{\beta}$ is

$$\text{var}\left[\hat{Y}(\mathbf{x}) \mid \mathbf{x}\right] = \mathbf{x}^T \text{var}\left[\hat{\beta}\right]\mathbf{x} = \sigma^2 \sum_{k=1}^{r} \frac{\delta_k^2}{(\delta_k^2+\lambda)^2}\left(\mathbf{v}_k^T\mathbf{x}\right)^2,$$

  which decreases as λ increases.

- Through the bias-variance trade-off, we hope to obtain better predictions in terms of expected conditional test error for an appropriate choice of λ.

Recall the expression of the expected conditional test error

$$\begin{aligned} Err(\mathbf{x}) &= \text{E}\left[(Y^* - \hat{Y})^2 \mid \mathbf{x}\right] \\ &= \text{var}\left[\hat{Y} \mid \mathbf{x}\right] + \text{bias}^2(\mathbf{x}) + \text{var}\left[Y^* \mid \mathbf{x}\right] \end{aligned}$$

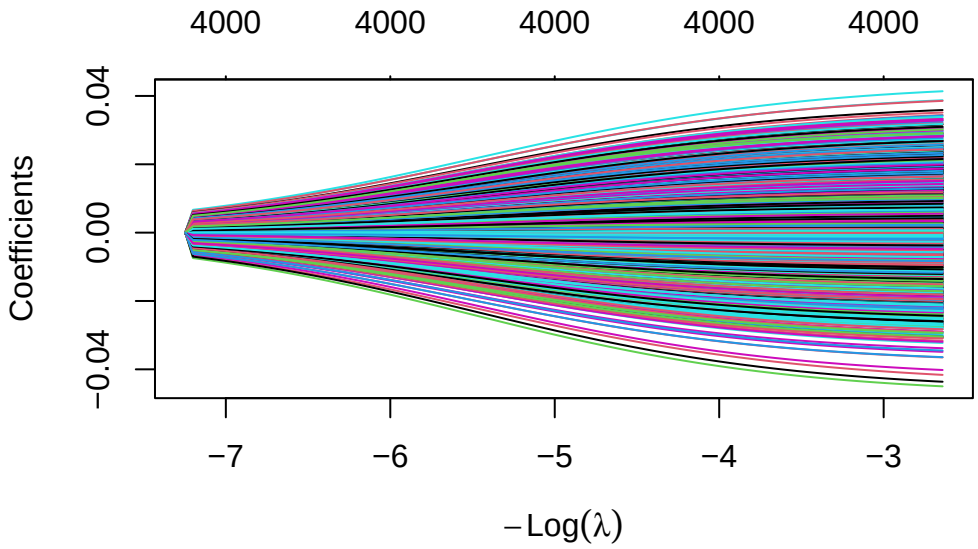
where

- $\hat{Y} = \hat{Y}(\mathbf{x}) = \mathbf{x}^T\hat{\beta}$ is the prediction at $\mathbf{x}$
- Y^* is an outcome at predictor $\mathbf{x}$
- $\mu(\mathbf{x}) = \text{E}\left[\hat{Y} \mid \mathbf{x}\right]$ and $\mu^*(x) = \text{E}\left[Y^* \mid \mathbf{x}\right]$
- $\text{bias}(\mathbf{x}) = \mu(\mathbf{x}) - \mu^*(\mathbf{x})$
- $\text{var}\left[Y^* \mid \mathbf{x}\right]$ the irreducible error that does not depend on the model. It simply originates from observations that randomly fluctuate around the true mean $\mu^*(x)$.

4.6 Toxicogenomics example

```
library(glmnet)
mRidge <- glmnet(
  x = toxData[,-1] |>
    as.matrix(),
  y = toxData |>
    pull(BA),
  alpha = 0) # ridge: alpha = 0

plot(mRidge, xvar="lambda")
```



The R function `glmnet` uses `lambda` to refer to the penalty parameter, and so do we in this course. Note, that λ is also often used for eigenvalues. To avoid confusion, we write the eigenvalues of $\mathbf{X}^T\mathbf{X}$ as δ_k^2, the squared singular values of $\mathbf{X}$.

The graph shows that with increasing penalty parameter, the parameter estimates are shrunken towards zero. The estimates will only reach zero for $\lambda \to \infty$. The stronger the shrinkage, the larger the bias (towards zero) and the smaller the variance of the parameter estimators (and hence also smaller variance of the predictions).

Another (informal) viewpoint is the following. By shrinking the estimates towards zero, the estimates lose some of their "degrees of freedom" so that the parameters become estimable with only $n < p$ data points. Even with a very small $\lambda > 0$, the parameters regain their estimability. However, estimable does not mean precise. Recall from Section 4.5 that

$$\text{var}\left[\hat{\beta}\right] = \sigma^2 \sum_{k=1}^{r} \frac{\delta_k^2}{(\delta_k^2 + \lambda)^2} \mathbf{v}_k \mathbf{v}_k^T.$$

For a small λ, the variance along directions $\mathbf{v}_k$ with small singular values is close to σ^2/δ_k^2, which is very large. The variance decreases as λ increases, and as $\lambda \to \infty$ the estimates converge to zero and no longer vary.

5 Lasso Regression

- The Lasso is another example of penalised regression.

- The lasso estimator of β is the solution to minimising the penalised SSE

$$\text{SSE}_{\text{pen}} = \sum_{i=1}^{n}(Y_i - \mathbf{x}_i^T\beta)^2 + \lambda\sum_{j=1}^{p}|\beta_j|.$$

or, equivalently, minimising

$$\text{SSE} = \|\mathbf{Y} - \mathbf{X}\|_2^2 \text{ subject to } \|\ \|_1 \leq c$$

with

- $\|\ \|_1 = \sum_{j=1}^{p}|\beta_j|$
- Despite strong similarity between ridge and lasso regression (L_2 versus L_1 norm in penalty term), there is no analytical solution of the lasso parameter estimator of .
- Fortunately, computational efficient algorithms have been implemented in statistical software
- The Lasso estimator of β is biased and generally has a smaller variance then the least-squares estimator.
- Hence, the bias-variance trade-off may here also help in finding better predictions with biased estimators.
- In contrast to ridge regression, however, the lasso estimator can give at most $\min(p, n)$ non-zero β-estimates.
- Hence, at first sight the lasso is not directly appropriate for high-dimensional settings.
- An important advantage of the lasso is that choosing an appropriate value for λ is a kind a model building or feature selection procedure (see further).

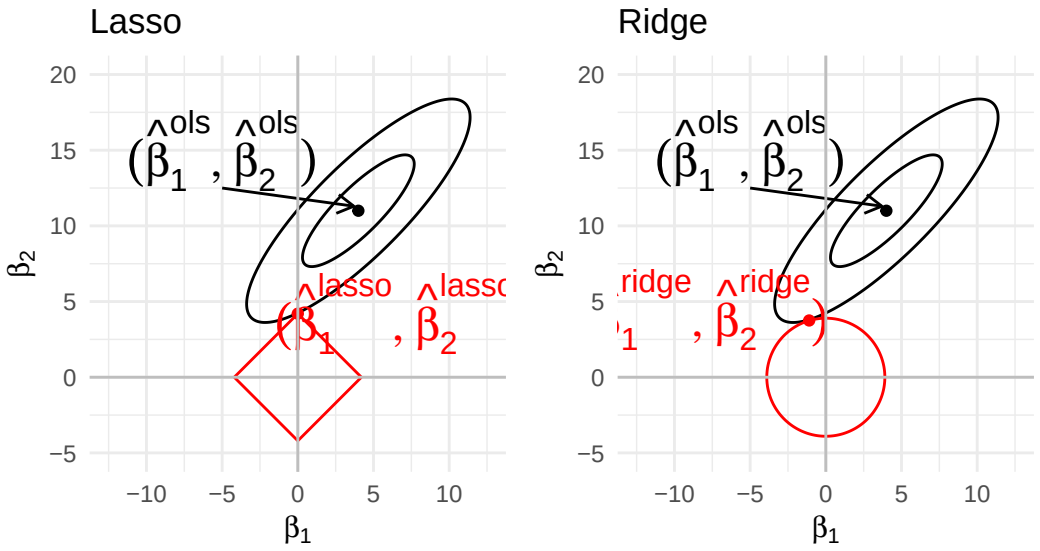
5.1 Graphical interpretation of Lasso vs ridge

Note that the lasso is a constrained regression problem with

$$\|\mathbf{Y} - \mathbf{X}\|_2^2 \text{ subject to } \|\ \|_1 \leq c$$

and ridge

$$\|\mathbf{Y} - \mathbf{X}\|_2^2 \text{ subject to } \|\ \|_2^2 \leq c$$

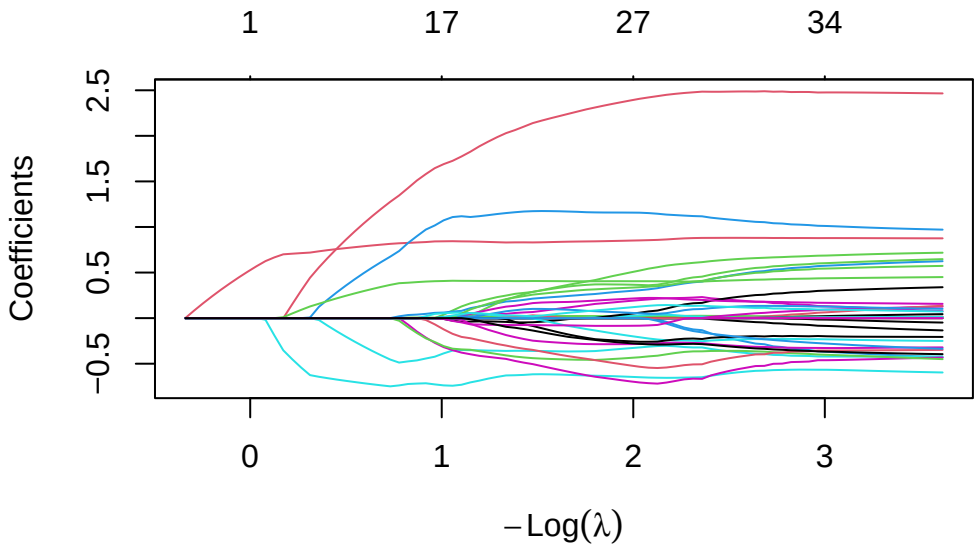


Note, that

- parameters for the lasso can never switch sign, they are set at zero! Selection!
- ridge regression can lead to parameters that switch sign.

5.2 Toxicogenomics example

```
mLasso <- glmnet(
  x = toxData[,-1] |>
    as.matrix(),
  y = toxData |>
    pull(BA),
alpha = 1)
plot(mLasso, xvar = "lambda")
```



- The graph with the paths of the parameter estimates nicely illustrates the typical behaviour of the lasso estimates as a function of λ: when λ increases the estimates are shrunken towards zero.
- When an estimate hits zero, it remains exactly equal to zero when γ further increases. A parameter estimate equal to zero, say $\widehat{\beta}_j = 0$, implies that the corresponding predictor x_j is no longer included in the model (i.e. $\beta_j x_j = 0$).
- The model fit is known as a sparse model fit (many zeroes). Hence, choosing a appropriate value for γ is like choosing the important predictors in the model (feature selection).

6 Splines and the connection to ridge regression.

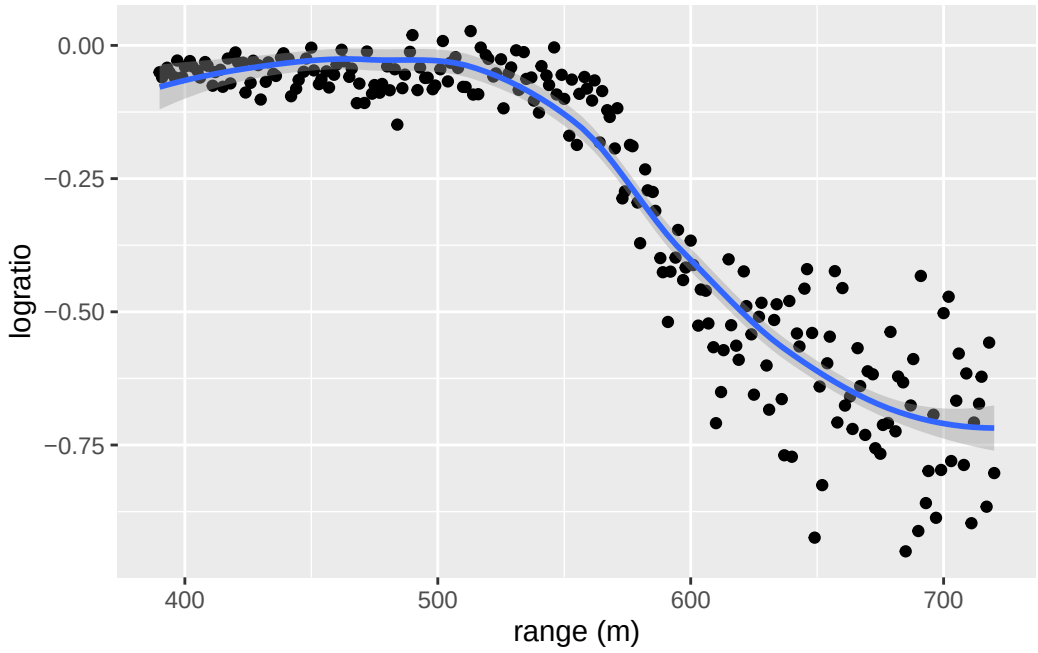
6.1 Lidar dataset

- LIDAR (light detection and ranging) uses the reflection of laser-emitted light to detect chemical compounds in the atmosphere.
- The LIDAR technique has proven to be an efficient tool for monitoring the distribution of several atmospheric pollutants of importance; see Sigrist (1994).
- The range is the distance traveled before the light is reflected back to its source.
- The logratio is the logarithm of the ratio of received light from two laser sources.

- One source had a frequency equal to the resonance frequency of the compound of interest, which was mercury in this study.
- The other source had a frequency off this resonance frequency.
- The concentration of mercury can be derived from a regression model of the logratio in function of the range for each range x.

```
library("SemiPar")
data(lidar)
pLidar <- lidar |>
  ggplot(aes(x = range, y = logratio)) +
  geom_point() +
  xlab("range (m)")

pLidar +
  geom_smooth()
```



- The data is non-linear
- Linear regression will not work!
- The data shows a smooth relation between the logratio and the range

6.2 Basis expansion

$$y_i = f(x_i) + \epsilon_i,$$

with

$$f(x) = \sum_{k=1}^{K} \theta_k b_k(x)$$

- Select set of basis functions $b_k(x)$
- Select number of basis functions K
- Examples
  - Polynomial model: x^k
  - Orthogonal series: Fourier, Legendre polynomials, Wavelets
  - Polynomial splines: $1, x, (x - t_m)_+$ with $m = 1, \ldots, K - 2$ knots t_m
  - ...

6.2.1 Truncated line basis

$$y_i = f(x_i) + \epsilon_i,$$

- One of the most simple basis expansions
- $f(x_i) = \beta_0 + \beta_1 x_i + \sum_{m=1}^{K-2} \theta_m (x_i - t_m)_+$ with $(.)_+$ the operator that takes the positive part.
- Note, that better basis expansions exist, which are orthogonal, computational more stable and/or continuous derivative beyond first order
- We will use this basis for didactical purposes
- We can use OLS to fit y w.r.t. the basis.

```
knots <- seq(400,700,12.5)

basis <- sapply(knots,
  function(k,y) (y-k)*(y>k),
  y= lidar |> pull(range)
  )

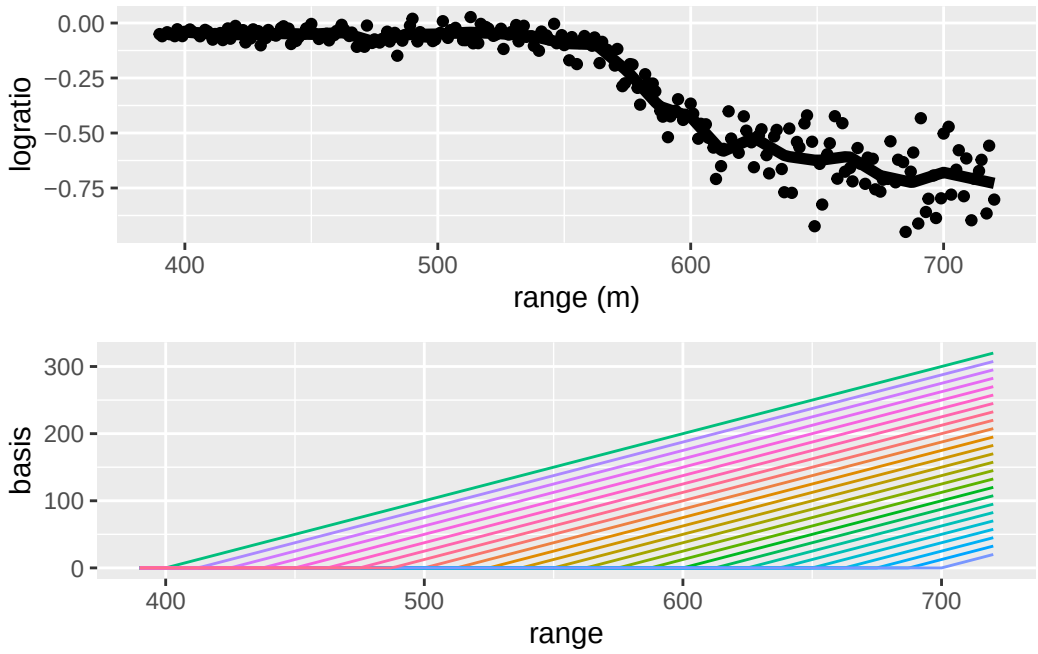
basisExp <- cbind(1, range = lidar |> pull(range), basis)

splineFitLs <- lm(logratio ~ -1 + basisExp, lidar)

pBasis <- basisExp[,-1] |>
  data.frame() |>
```

```
  gather("basis","values",-1) |>
  ggplot(aes(x = range, y = values, color = basis)) +
  geom_line() +
  theme(legend.position="none") +
  ylab("basis")

grid.arrange(
  pLidar +
    geom_line(aes(x = lidar$range, y = splineFitLs$fitted), lwd = 2),
  pBasis,
  ncol=1)
```



- Note, that the model is overfitting!
- The fit is very wiggly and is tuned too much to the data.
- The fit has a large variance and low bias.
- It will therefore not generalise well to predict the logratio of future observations.

6.2.1.1 Solution for overfitting?

- We could perform model selection on the basis to select the important basis functions to model the signal. But, this will have the undesired property that the fit will no longer be smooth.

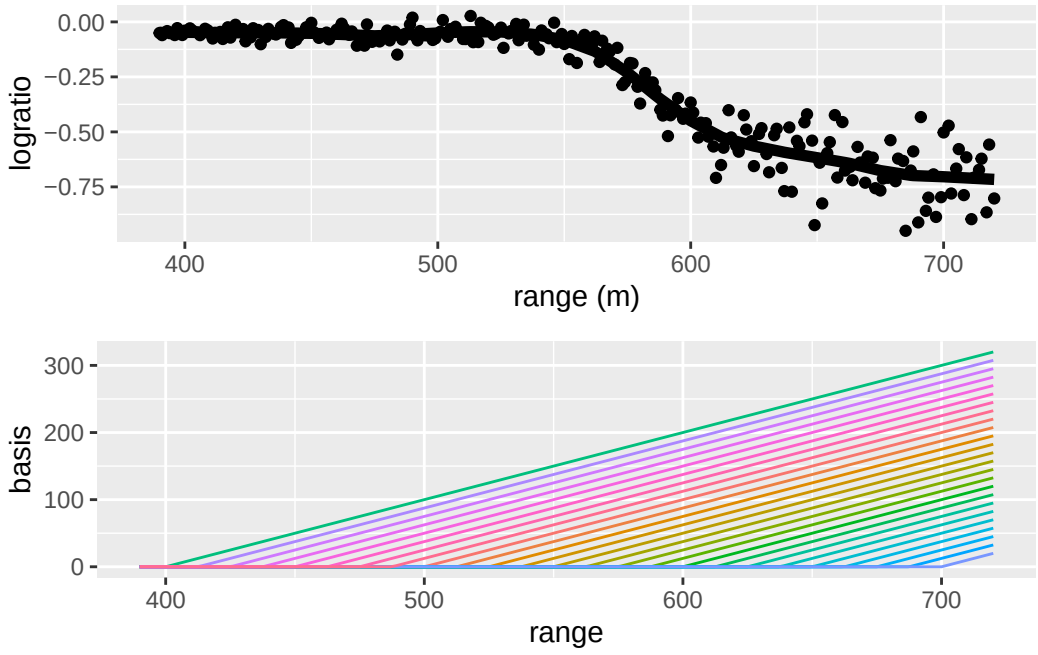
- We can also adopt a ridge penalty!
- However, we do not want to penalise the intercept and the linear term.
- Ridge criterion

$$\| \mathbf{Y} - \mathbf{X} \|^2 + \lambda \beta^T \mathbf{D} \beta$$

With $\mathbf{D}$ with dimensions (K,K): $\mathbf{D} = \begin{bmatrix} \mathbf{0}_{2\times 2} & \mathbf{0}_{2\times K-2} \\ \mathbf{0}_{K-2\times 2} & \mathbf{I}_{K-2\times K-2} \end{bmatrix}$

- Here we will set the penalty at 900.

```
D <- diag(ncol(basisExp))
D[1:2,1:2] <- 0
lambda <- 900
betaRidge <- solve(t(basisExp)%*%basisExp+(lambda*D))%*%t(basisExp)%*%lidar$logratio
grid.arrange(
  pLidar +
    geom_line(aes(x = lidar$range, y = c(basisExp %*% betaRidge)), lwd = 2),
  pBasis,
  ncol=1)
```



How do we choose λ?

7 Evaluation of Prediction Models

Predictions are calculated with the fitted model

$$\widehat{Y}(\mathbf{x}) = \widehat{m}(\mathbf{x}) = \mathbf{x}^T\widehat{\beta}$$

when focussing on prediction, we want the prediction error to be as small as possible.

The **prediction error** for a prediction at covariate pattern $\mathbf{x}$ is given by

$$Y^* - \widehat{Y}(\mathbf{x}),$$

where

- $\widehat{Y}(\mathbf{x}) = \mathbf{x}^T\widehat{\beta}$ is the prediction at $\mathbf{x}$
- Y^* is an outcome at covariate pattern $\mathbf{x}$

Prediction is typically used to predict an outcome before it is observed.

- Hence, the outcome Y^* is not observed yet, and
- the prediction error cannot be computed.

- Recall that the prediction model $\widehat{Y}(\mathbf{x})$ is estimated by using data in the training data set $(\mathbf{X}, \mathbf{Y})$, and
- that the outcome Y^* is an outcome at $\mathbf{x}$ which is assumed to be independent of the training data.
- Goal is to use prediction model for predicting a future observation (Y^*), i.e. an observation that still has to be realised/observed (otherwise prediction seems rather useless).
- Hence, Y^* can never be part of the training data set.

Here we provide definitions and we show how the prediction performance of a prediction model can be evaluated from data.

Let $\mathcal{T} = (\mathbf{Y}, \mathbf{X})$ denote the training data, from which the prediction model $\widehat{Y}(\cdot)$ is build. This building process typically involves feature selection and parameter estimation.

We will use a more general notation for the prediction model: $\widehat{m}(\mathbf{x}) = \widehat{Y}(\mathbf{x})$.

7.1 Test or Generalisation Error

The test or generalisation error for prediction model $\hat{m}(\cdot)$ is given by

$$\mathrm{Err}_{\mathcal{T}} = \mathrm{E}_{Y^*, X^*}\left[(Y^* - \hat{m}(\mathbf{X}^*))^2 \mid \mathcal{T}\right]$$

where (Y^*, X^*) is independent of the training data.

- Note that the test error is conditional on the training data $\mathcal{T}$.
- Hence, the test error evaluates the performance of the single model build from the observed training data.
- This is the ultimate target of the model assessment, because it is exactly this prediction model that will be used in practice and applied to future predictors $\mathbf{X}^*$ to predict Y^*.
- The test error is defined as an average over all such future observations $(Y^*, \mathbf{X}^*)$.

7.2 Conditional test error

Sometimes the conditional test error is used:

The conditional test error in $\mathbf{x}$ for prediction model $\hat{m}(\mathbf{x})$ is given by

$$\mathrm{Err}_{\mathcal{T}}(\mathbf{x}) = \mathrm{E}_{Y^*}\left[(Y^* - \hat{m}(\mathbf{x}))^2 \mid \mathcal{T}, \mathbf{x}\right]$$

where Y^* is an outcome at predictor $\mathbf{x}$, independent of the training data.

Hence,

$$\mathrm{Err}_{\mathcal{T}} = \mathrm{E}_{X^*}\left[\mathrm{Err}_{\mathcal{T}}(\mathbf{X}^*)\right].$$

A closely related error is the **insample error**.

7.3 Insample Error

The insample error for prediction model $\widehat{m}(\mathbf{x})$ is given by

$$\mathrm{Err}_{\mathrm{in}\mathcal{T}} = \frac{1}{n}\sum_{i=1}^{n}\mathrm{Err}_{\mathcal{T}}(\mathbf{x}_i),$$

i.e. the insample error is the sample average of the conditional test errors evaluated in the n training dataset predictors $\mathbf{x}_i$.

Since $\mathrm{Err}_{\mathcal{T}}$ is an average over all $\mathbf{X}$, even over those predictor patterns not observed in the training dataset, it is sometimes referred to as the **outsample error**.

7.4 Estimation of the insample error

We start with introducing the training error, which is closely related to the MSE in linear models.

7.4.1 Training error

The training error is given by

$$\overline{\mathrm{err}} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \widehat{m}(\mathbf{x}_i))^2,$$

where the $(Y_i, \mathbf{x}_i)$ from the training dataset which is also used for the calculation of $\widehat{m}$.

- The training error is an overly optimistic estimate of the test error $\mathrm{Err}_{\mathcal{T}}$.
- The training error will never increase when the model becomes more complex. $\longrightarrow$ cannot be used directly as a model selection criterion.

Indeed, model parameters are often estimated by minimising the training error (cfr. SSE).

- Hence the fitted model adapts to the training data, and
- training error will be an overly optimistic estimate of the test error $\mathrm{Err}_{\mathcal{T}}$.

It can be shown that the training error is related to the insample test error via

$$\mathrm{E}_{\mathbf{Y}}\left[\mathrm{Err}_{\mathrm{in}\mathcal{T}}\right] = \mathrm{E}_{\mathbf{Y}}\left[\overline{\mathrm{err}}\right] + \frac{2}{n}\sum_{i=1}^{n}\mathrm{cov}_{\mathbf{Y}}\left[\hat{m}(\mathbf{x}_i), Y_i\right],$$

where $\mathbf{Y} = (Y_1, \ldots, Y_n)^T$ are the outcomes of the training data $\mathcal{T} = (\mathbf{Y}, \mathbf{X})$.

- $\mathrm{E}_{\mathbf{Y}}$ averages over repeated training datasets with the same predictors $\mathbf{x}_1, \ldots, \mathbf{x}_n$ (the design is kept fixed), but with new random outcomes $\mathbf{Y}$.
- This expectation is needed because $\overline{\mathrm{err}}$ and $\mathrm{Err}_{\mathrm{in}\mathcal{T}}$ are random variables: they depend on the training data through $\hat{m}$. The identity does not hold for a single training dataset, only on average.
- Do not confuse the two kinds of outcomes: the training outcomes Y_i are used to fit $\hat{m}$ and are averaged out by $\mathrm{E}_{\mathbf{Y}}$, while the new outcomes Y_i^* at the same $\mathbf{x}_i$ are independent of the training data and are already averaged out in the definition of $\mathrm{Err}_{\mathcal{T}}(\mathbf{x}_i)$.

7.4.2 Proof

Write $\mu_i = \mathrm{E}[Y_i] = \mathrm{E}[Y_i^*]$, $\mathrm{var}[Y_i] = \mathrm{var}[Y_i^*] = \sigma^2$ and $\hat{m}_i = \hat{m}(\mathbf{x}_i)$. Subtracting and adding μ_i gives

$$\begin{aligned} Y_i^* - \hat{m}_i &= (Y_i^* - \mu_i) - (\hat{m}_i - \mu_i) \\ Y_i - \hat{m}_i &= (Y_i - \mu_i) - (\hat{m}_i - \mu_i). \end{aligned}$$

Squaring and taking expectations gives

$$\begin{aligned} \mathrm{E}\left[(Y_i^* - \hat{m}_i)^2\right] &= \sigma^2 + \mathrm{E}\left[(\hat{m}_i - \mu_i)^2\right] - 2\,\mathrm{cov}\left[Y_i^*, \hat{m}_i\right] \\ \mathrm{E}\left[(Y_i - \hat{m}_i)^2\right] &= \sigma^2 + \mathrm{E}\left[(\hat{m}_i - \mu_i)^2\right] - 2\,\mathrm{cov}\left[Y_i, \hat{m}_i\right]. \end{aligned}$$

The first two terms are identical. Only the covariance differs:

- Y_i^* is independent of the training data, so $\mathrm{cov}\left[Y_i^*, \hat{m}_i\right] = 0$;
- Y_i is used to fit $\hat{m}$, so $\mathrm{cov}\left[Y_i, \hat{m}_i\right]$ is typically positive.

Subtracting both expressions and averaging over $i = 1, \ldots, n$ gives the identity.

Note, that for linear models

$$\hat{m}(\mathbf{x}_i) = \mathbf{X}\hat{\beta} = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y} = \mathbf{H}\mathbf{Y}$$

with

- **H** the hat matrix and
- all Y_i are assumed to be independently distributed with mean $\mathbf{x}_i^T\beta$ and variance σ^2 (normality is not required)

Hence, for linear models with independent observations

$$\begin{aligned}
\mathrm{cov}_{\mathbf{Y}}\left[\widehat{m}(\mathbf{x}_i), Y_i\right] &= \mathrm{cov}_{\mathbf{Y}}\left[\mathbf{H}_i^T\mathbf{Y}, Y_i\right] = \mathrm{cov}_{\mathbf{Y}}\left[\sum_{j=1}^{n} h_{ij}Y_j, Y_i\right] && (1)\\
&= \mathrm{cov}_{\mathbf{Y}}\left[h_{ii}Y_i, Y_i\right] \qquad (\text{the } Y_j, j \neq i, \text{ are independent of } Y_i) && (2)\\
&= h_{ii}\mathrm{cov}_{\mathbf{Y}}\left[Y_i, Y_i\right] && (3)\\
&= h_{ii}\sigma^2 && (4)\\
& && (5)
\end{aligned}$$

And we can thus estimate the insample error by Mallows' C_p

$$\begin{aligned}
C_p &= \overline{\mathrm{err}} + \frac{2\sigma^2}{n}\mathrm{tr}(\mathbf{H}) && (6)\\
&= \overline{\mathrm{err}} + \frac{2\sigma^2 p}{n} && (7)
\end{aligned}$$

with p the number of predictors.

- Mallows' C_p is often used for model selection.
- Note, that we can also consider it as a kind of penalized least squares:

$$n \times C_p = \|\mathbf{Y} - \mathbf{X}\beta\|_2^2 + 2\sigma^2\|\beta\|_0$$

with L_0 norm $\|\beta\|_0 = \sum_{j=1}^{p}\beta_j^0 = p$.

7.5 Expected test error

The test or generalisation error was defined conditionally on the training data. By averaging over the distribution of training datasets, the expected test error arises.

$$\begin{aligned}
\mathrm{E}_{\mathcal{T}}\left[\mathrm{Err}_{\mathcal{T}}\right] &= \mathrm{E}_{\mathcal{T}}\left[\mathrm{E}_{Y^*,X^*}\left[(Y^* - \widehat{m}(\mathbf{X}^*))^2 \mid \mathcal{T}\right]\right]\\
&= \mathrm{E}_{Y^*,X^*,\mathcal{T}}\left[(Y^* - \widehat{m}(\mathbf{X}^*))^2\right].
\end{aligned}$$

- The expected test error may not be of direct interest when the goal is to assess the prediction performance of a single prediction model $\hat{m}(\cdot)$.
- The expected test error averages the test errors of all models that can be build from all training datasets, and hence this may be less relevant when the interest is in evaluating one particular model that resulted from a single observed training dataset.
- Also note that building a prediction model involves both parameter estimation and feature selection.
- Hence the expected test error also evaluates the feature selection procedure (on average).
- If the expected test error is small, it is an indication that the model building process gives good predictions for future observations $(Y^*, \mathbf{X}^*)$ on average.

7.5.1 Estimating the Expected test error

The expected test error may be estimated by cross validation (CV).

7.5.1.1 Leave one out cross validation (LOOCV)}

The LOOCV estimator of the expected test error (or expected outsample error) is given by

$$\text{CV} = \frac{1}{n}\sum_{i=1}^{n}\left(Y_i - \hat{m}^{-i}(\mathbf{x}_i)\right)^2,$$

where

- the $(Y_i, \mathbf{x}_i)$ form the training dataset
- $\hat{m}^{-i}$ is the fitted model based on all training data, except observation i
- $\hat{m}^{-i}(\mathbf{x}_i)$ is the prediction at $\mathbf{x}_i$, which is the observation left out the training data before building model m.

Some rationale as to why LOOCV offers a good estimator of the outsample error:

- the prediction error $Y^* - \hat{m}(\mathbf{x})$ is mimicked by not using one of the training outcomes Y_i for the estimation of the model so that this Y_i plays the role of Y^*, and, consequently, the fitted model $\hat{m}^{-i}$ is independent of Y_i
- the sum in CV is over all $\mathbf{x}_i$ in the training dataset, but each term $\mathbf{x}_i$ was left out once for the calculation of $\hat{m}^{-i}$. Hence, $\hat{m}^{-i}(\mathbf{x}_i)$ mimics an outsample prediction.
- the sum in CV is over n different training datasets (each one with a different observation removed), and hence CV is an estimator of the *expected* test error.
- For linear models the LOOCV can be readily obtained from the fitted model: i.e.

$$\text{CV} = \frac{1}{n}\sum_{i=1}^{n}\frac{e_i^2}{(1-h_{ii})^2}$$

with e_i the residuals from the model that is fitted based on all training data.

An alternative to LOOCV is the k-fold cross validation procedure. It also gives an estimate of the expected outsample error.

7.5.1.2 k-fold cross validation

- Randomly divide the training dataset into k approximately equal subsets . Let S_j denote the index set of the jth subset (referred to as a **fold**). Let n_j denote the number of observations in fold j.
- The k-fold cross validation estimator of the expected outsample error is given by

$$\text{CV}_k = \frac{1}{k}\sum_{j=1}^{k}\frac{1}{n_j}\sum_{i\in S_j}\left(Y_i - \widehat{m}^{-S_j}(\mathbf{x}_i)\right)^2$$

  where $\widehat{m}^{-S_j}$ is the model fitted using all training data, except observations in fold j (i.e. observations $i \in S_j$).

The cross validation estimators of the expected outsample error are nearly unbiased. One argument that helps to understand where the bias comes from is the fact that e.g. in de LOOCV estimator the model is fit on only $n-1$ observations, whereas we are aiming at estimating the outsample error of a model fit on all n training observations. Fortunately, the bias is often small and is in practice hardly a concern.

k-fold CV is computationally more complex.

Since CV and CV_k are estimators, they also show sampling variability. Standard errors of the CV or CV_k can be computed. We don't show the details, but in the example this is illustrated.

7.5.2 Bias Variance trade-off

For the expected conditional test error in $\mathbf{x}$, it holds that

$$\begin{aligned} \mathrm{E}_{\mathcal{T}}\left[\mathrm{Err}_{\mathcal{T}}(\mathbf{x})\right] &= \mathrm{E}_{Y^*,\mathcal{T}}\left[(Y^* - \widehat{m}(\mathbf{x}))^2 \mid \mathbf{x}\right] \\ &= \mathrm{var}_{\mathbf{Y}}\left[\widehat{Y}(\mathbf{x}) \mid \mathbf{x}\right] + (\mu(\mathbf{x}) - \mu^*(\mathbf{x}))^2 + \mathrm{var}_{Y^*}\left[Y^* \mid \mathbf{x}\right] \end{aligned}$$

where $\mu(\mathbf{x}) = \mathrm{E}_{\mathbf{Y}}\left[\widehat{Y}(\mathbf{x}) \mid \mathbf{x}\right]$ and $\mu^*(\mathbf{x}) = \mathrm{E}_{Y^*}\left[Y^* \mid \mathbf{x}\right]$.

- **bias**: $\mathrm{bias}(\mathbf{x}) = \mu(\mathbf{x}) - \mu^*(\mathbf{x})$
- $\mathrm{var}_{Y^*}\left[Y^* \mid \mathbf{x}\right]$ does not depend on the model, and is referred to as the **irreducible variance**.

The importance of the bias-variance trade-off can be seen from a model selection perspective. When we agree that a good model is a model that has a small expected conditional test error at some point $\mathbf{x}$, then the bias-variance trade-off shows us that a model may be biased as long as it has a small variance to compensate for the bias. It often happens that a biased model has a substantial smaller variance. When these two are combined, a small expected test error may occur.

Also note that the model m which forms the basis of the prediction model $\widehat{m}(\mathbf{x})$ does NOT need to satisfy $m(\mathbf{x}) = \mu(\mathbf{x})$ or $m(\mathbf{x}) = \mu^*(\mathbf{x})$. The model m is known by the data-analyst (its the basis of the prediction model), whereas $\mu(\mathbf{x})$ and $\mu^*(\mathbf{x})$ are generally unknown to the data-analyst. We only hope that m serves well as a prediction model.

7.5.3 In practice

We use cross validation to estimate the lambda penalty for penalised regression:

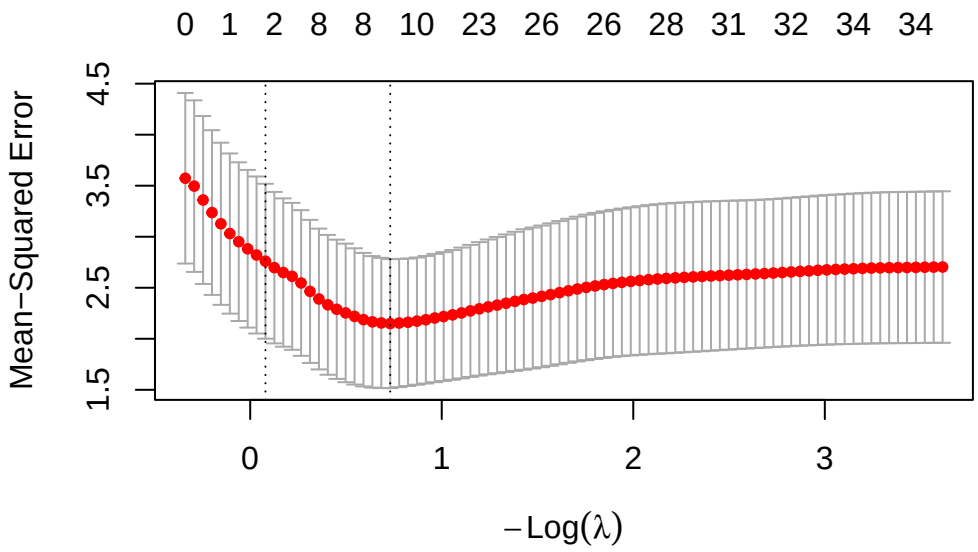
- Ridge Regression
- Lasso
- Build models, e.g. select the number of PCs for PCA regression
- Splines

7.5.4 Toxicogenomics example

7.5.4.1 Lasso

```
set.seed(15)
library(glmnet)
mCvLasso <- cv.glmnet(
  x = toxData[,-1] |>
    as.matrix(),
  y = toxData |>
    pull(BA),
  alpha = 1)  # lasso alpha=1

plot(mCvLasso)
```



Default CV procedure in `cv.glmnet` is $k = 10$-fold CV.

The Graphs shows

- 10-fold CV estimates of the extra-sample error as a function of the lasso penalty parameter λ.
- estimate plus and minus once the estimated standard error of the CV estimate (grey bars)
- On top the number of non-zero regression parameter estimates are shown.

Two vertical reference lines are added to the graph. They correspond to

- the $\log(\lambda)$ that gives the smallest CV estimate of the extra-sample error, and
- the largest $\log(\lambda)$ that gives a CV estimate of the extra-sample error that is within one standard error from the smallest error estimate.
- The latter choice of λ has no firm theoretical basis, except that it somehow accounts for the imprecision of the error estimate. One could loosely say that this γ corresponds to the smallest model (i.e. least number of predictors) that gives an error that is within margin of error of the error of the best model.

```
mLassoOpt <- glmnet(
  x = toxData[,-1] |>
    as.matrix(),
  y = toxData |>
    pull(BA),
    alpha = 1,
    lambda = mCvLasso$lambda.min)

summary(coef(mLassoOpt))
```

```
#> 4001 x 1 sparse Matrix of class "dgCMatrix", with 9 entries
#>      i j             x
#> 1    1 1  2.135417e-17
#> 2    7 1  7.723665e-01
#> 3  105 1  6.745830e-01
#> 4  147 1 -7.479963e-01
#> 5  420 1  1.275535e+00
#> 6  453 1  4.272541e-02
#> 7 1720 1 -4.548090e-01
#> 8 1952 1  3.651340e-01
#> 9 2032 1  4.115143e-16
```

With the optimal λ (smallest error estimate) the output shows the 9 non-zero estimated regression coefficients (sparse solution).

```
mLasso1se <- glmnet(
  x = toxData[,-1] |>
    as.matrix(),
    y= toxData |>
      pull(BA),
    alpha = 1,
    lambda = mCvLasso$lambda.1se)

mLasso1se |>
  coef() |>
  summary()
```

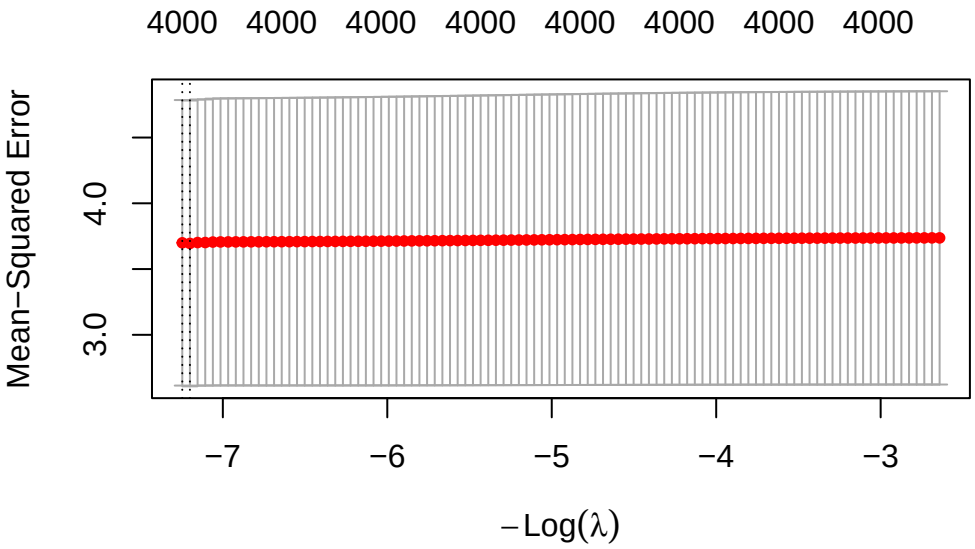
```
#> 4001 x 1 sparse Matrix of class "dgCMatrix", with 3 entries
#>     i j             x
#> 1   1 1  3.608536e-18
#> 2   7 1  6.255918e-01
#> 3 147 1 -1.767770e-02
```

This shows the solution for the largest λ within one standard error of the optimal model. Now only 3 non-zero estimates result.

7.5.4.2 Ridge

```
mCvRidge <- cv.glmnet(
  x = toxData[,-1] |>
    as.matrix(),
    y = toxData |>
      pull(BA),
      alpha = 0)  # ridge alpha=0

plot(mCvRidge)
```



- Ridge does not seem to have optimal solution.
- 10-fold CV is also larger than for lasso.

7.5.4.3 PCA regression

```
set.seed(1264)
library(DAAG)

tox <- data.frame(
  Y = toxData |>
    pull(BA),
  PC = Zk)

PC.seq <- 1:25
Err <- numeric(25)

mCvPca <- cv.lm(
  Y~PC.1,
  data = tox,
  m = 5,
```

```
  printit = FALSE)

Err[1]<-attr(mCvPca,"ms")

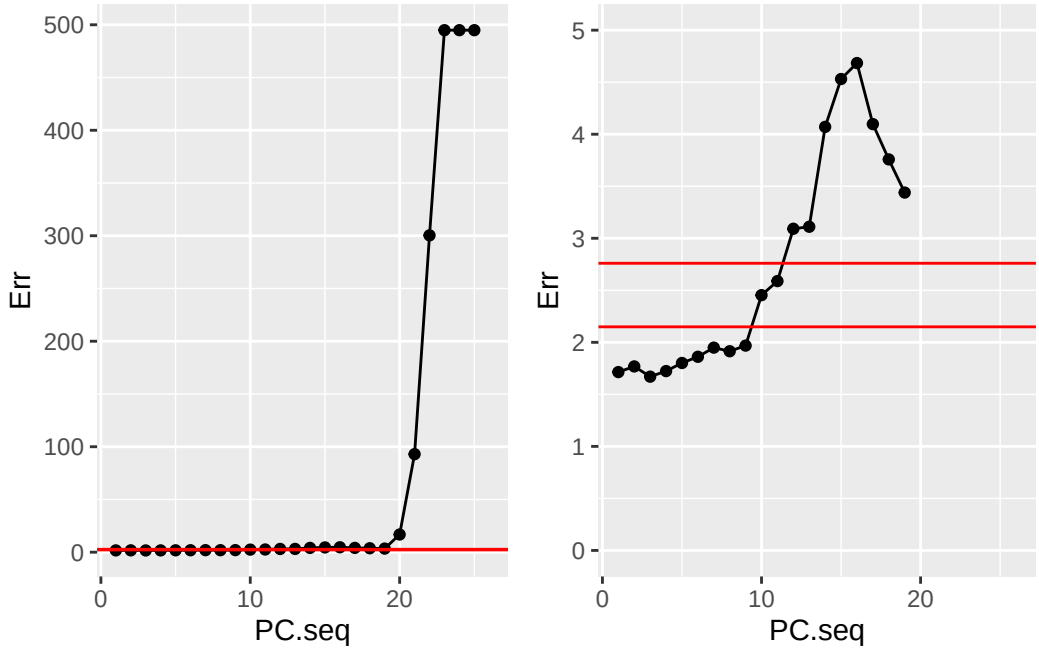
for(i in 2:25) {
  mCvPca <- cv.lm(
    as.formula(
      paste("Y ~ PC.1 + ",
        paste("PC.", 2:i, collapse = "+", sep=""),
        sep=""
      )
    ),
    data = tox,
    m = 5,
    printit = FALSE)
  Err[i]<-attr(mCvPca,"ms")
}
```

- Here we illustrate principal component regression.
- The most important PCs are selected in a forward model selection procedure.
- Within the model selection procedure the models are evaluated with 5-fold CV estimates of the outsample error.
- It is important to realise that a forward model selection procedure will not necessarily result in the best prediction model, particularly because the order of the PCs is generally not related to the importance of the PCs for predicting the outcome.
- A supervised PC would be better.

```
pPCreg <- data.frame(PC.seq, Err) |>
  ggplot(aes(x = PC.seq, y = Err)) +
  geom_line() +
  geom_point() +
  geom_hline(
    yintercept = c(
      mCvLasso$cvm[mCvLasso$lambda==mCvLasso$lambda.min],
      mCvLasso$cvm[mCvLasso$lambda==mCvLasso$lambda.1se]),
    col = "red") +
  xlim(1,26)

grid.arrange(
  pPCreg,
```

```
pPCreg + ylim(0,5),
ncol=2)
```



- The graph shows the CV estimate of the outsample error as a function of the number of sparse PCs included in the model.
- A very small error is obtained with the model with only the first PC. The best model with 3 PCs.
- The two vertical reference lines correspond to the error estimates obtained with lasso (optimal λ and largest λ within one standard error).
- Thus although there was a priori no guarantee that the first PCs are the most predictive, it seems to be the case here (we were lucky!).
- Moreover, the first PC resulted in a small outsample error.
- Note that the graph does not indicate the variability of the error estimates (no error bars).
- Also note that the graph clearly illustrates the effect of overfitting: including too many PCs causes a large outsample error.

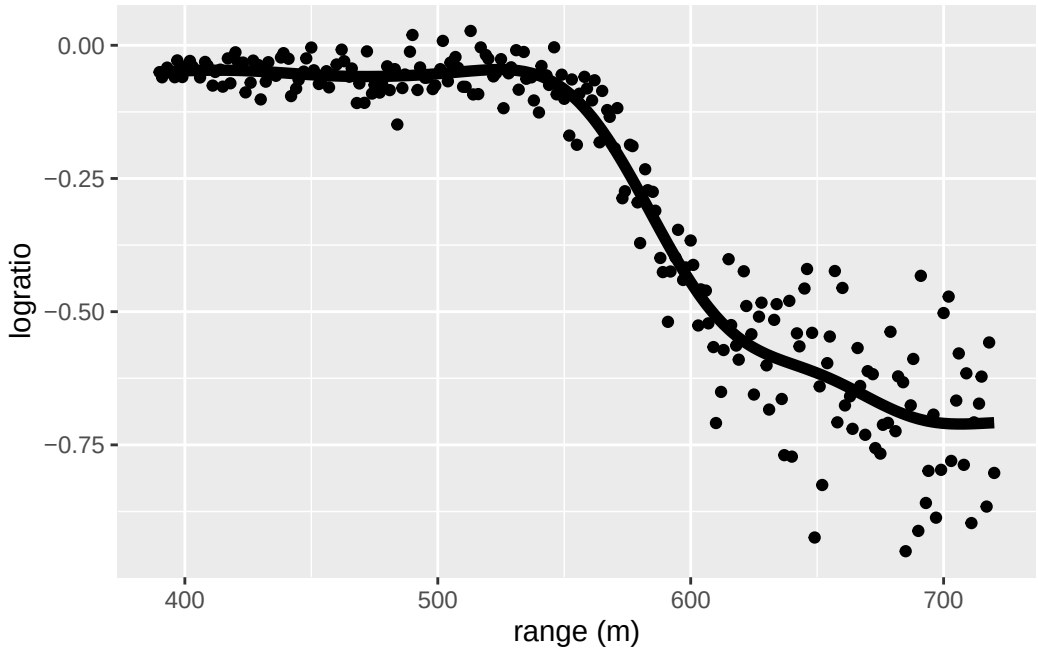
7.5.5 Lidar Example: splines

- We use the mgcv package to fit the spline model to the lidar data.
- A better basis is used than the truncated spline basis
- Thin plate splines are also linear smoothers, i.e. $\hat{Y} = \hat{m}(\mathbf{X}) = \mathbf{SY}$
- So their variance can be easily calculated.
- The ridge/smoothness penalty is chosen by generalized cross validation.

```
library(mgcv)
gamfit <- gam(logratio ~ s(range), data = lidar)
gamfit$sp
```

```
#>    s(range)
#> 0.006114634
```

```
pLidar +
  geom_line(aes(x = lidar$range, y = gamfit$fitted), lwd = 2)
```



7.6 More general error definitions

So far we only looked at continuous outcomes Y and errors defined in terms of the squared loss $(Y^* - \hat{m}(\mathbf{x}))^2$.

More generally, a **loss function** measures an discrepancy between the prediction $\hat{m}(\mathbf{x})$ and an independent outcome Y^* that corresponds to $\mathbf{x}$.

Some examples for continuous Y:

$$\begin{aligned} L(Y^*, \hat{m}(\mathbf{x})) &= (Y^* - \hat{m}(\mathbf{x}))^2 \text{ (squared error)} \\ L(Y^*, \hat{m}(\mathbf{x})) &= |Y^* - \hat{m}(\mathbf{x})| \text{ (absolute error)} \\ L(Y^*, \hat{m}(\mathbf{x})) &= 2\left[\log f(Y^*; Y^*) - \log f(Y^*; \hat{m}(\mathbf{x}))\right] \text{ (deviance).} \end{aligned}$$

In the expression of the deviance

- $f(y; \mu)$ denotes the density function of the assumed outcome distribution with mean μ,
- $f(Y^*; Y^*)$ is the density evaluated with the mean set to the outcome itself (cfr. perfect fit, saturated model), and
- $f(Y^*; \hat{m}(\mathbf{x}))$ is the density of the same distribution but with mean set to the predicted outcome $\hat{m}(\mathbf{x})$.

With a given loss function, the errors are defined as follows:

- Test or generalisation or outsample error

$$\text{Err}_{\mathcal{T}} = \text{E}_{Y^*, X^*}\left[L(Y^*, \hat{m}(\mathbf{X}^*))\right]$$

- Training error

$$\overline{\text{err}} = \frac{1}{n}\sum_{i=1}^{n} L(Y_i, \hat{m}(\mathbf{x}_i))$$

- ...

When an exponential family distribution is assumed for the outcome distribution, and when the deviance loss is used, the insample error can be estimated by means of the AIC.

7.6.1 Akaike's Information Criterion (AIC)

The AIC for a model m is given by

$$\text{AIC} = -2 \log \hat{L}(m) + 2p$$

where $\hat{L}(m)$ is the maximised likelihood for model m.

The AIC is an estimate of n times the insample error under the deviance loss. As for Mallows' C_p, the insample error is estimated as the training error plus a correction for its optimism.

- **Training error.** With the deviance loss

$$n\,\overline{\text{err}} = \sum_{i=1}^{n} 2\left[\log f(Y_i; Y_i) - \log f(Y_i; \hat{m}(\mathbf{x}_i))\right] = -2 \log \hat{L}(m) + \text{constant},$$

  where the constant comes from the saturated model and does not depend on model m. The training error is too optimistic because the p parameters are estimated by maximising exactly this likelihood.

- It can be shown that the gap is approximately $2p$ on the $-2 \log L$ scale, so

$$n\,\text{E}_{\mathbf{Y}}\left[\text{Err}_{\text{in}\mathcal{T}}\right] \approx \text{E}_{\mathbf{Y}}\left[-2 \log \hat{L}(m)\right] + 2p + \text{constant} = \text{E}_{\mathbf{Y}}\left[\text{AIC}\right] + \text{constant}.$$

When assuming normally distributed error terms and homoscedasticity,

$$-2 \log L(\beta, \sigma^2) = n \log(2\pi\sigma^2) + \frac{\text{SSE}(m)}{\sigma^2},$$

with $\text{SSE}(m)$ the residual sum of squares of model m.

- σ^2 **known**: the first term is a constant and $-2 \log \hat{L}(m) = \text{SSE}(m)/\sigma^2 + \text{constant}$, so

$$\text{AIC} = \frac{\text{SSE}(m)}{\sigma^2} + 2p + \text{constant} = \frac{n}{\sigma^2}\left[\overline{\text{err}}(m) + \frac{2\sigma^2 p}{n}\right] + \text{constant} = \frac{n}{\sigma^2} C_p + \text{constant},$$

  i.e. AIC and C_p select the same model.

- σ^2 **unknown**: plugging in its maximum likelihood estimator $\hat{\sigma}^2 = \text{SSE}(m)/n$ gives

$$-2 \log \hat{L}(m) = n \log\left(2\pi \frac{\text{SSE}(m)}{n}\right) + n = n \log \text{SSE}(m) + \text{constant},$$

  so the log of the SSE originates from the estimation of σ^2, and the AIC becomes

$$\text{AIC} = n \log \text{SSE}(m) + 2p = n \log(n\overline{\text{err}}(m)) + 2p,$$

  up to a constant. Strictly, σ^2 is an additional parameter and the penalty should be $2(p+1)$, but this extra term is the same for all models and does not affect model selection.

Note, that for unknown σ^2, Mallows' C_p also arises as a linearised version of the AIC. A first order Taylor expansion of $\log \overline{\text{err}}$ around σ^2, $\log \overline{\text{err}} \approx \log \sigma^2 + (\overline{\text{err}} - \sigma^2)/\sigma^2$, gives, with $n \log n$ the same for all models,

$$\begin{aligned}
\text{AIC} &= n \log(n\overline{\text{err}}(m)) + 2p = n \log n + n \log \overline{\text{err}}(m) + 2p \\
&\approx \frac{n}{\sigma^2} \left[\overline{\text{err}}(m) + \frac{2\sigma^2 p}{n} \right] + \text{constant} \\
&= \frac{n}{\sigma^2} C_p + \text{constant}.
\end{aligned}$$

In linear models with normal error terms, C_p is an unbiased estimator of the insample error.

In summary, both criteria estimate the insample error as the training error plus a correction for its optimism:

Model	Loss	$n\times$ training error	$n\times$ optimism	Criterion
Linear model	squared error	SSE	$2\sigma^2 \operatorname{tr}(\mathbf{H}) = 2\sigma^2 p$	Mallows' C_p
Likelihood model	deviance	$-2 \log \hat{L}(m) +$ constant	$2p$	AIC

7.6.2 Bayesian Information Criterion (BIC)

The BIC for a model m is given by

$$\text{BIC} = -2 \log \hat{L}(m) + p \log(n)$$

where $\hat{L}(m)$ is the maximised likelihood for model m.

The BIC has a similar form as the AIC, but a different origin: it is derived from Bayesian model selection and approximates the posterior probability of model m. It is therefore not an estimator of the insample error. We will not elaborate further on its derivation in this course.

When assuming normally distributed error terms and homoscedasticity, the BIC becomes

$$\text{BIC} = n \log \text{SSE}(m) + p \log(n) = n \log(n\overline{\text{err}}(m)) + p \log(n)$$

with $\text{SSE}(m)$ the residual sum of squares of model m. As for the AIC, the $n \log \text{SSE}(m)$ term originates from plugging in the maximum likelihood estimator $\hat{\sigma}^2 = \text{SSE}(m)/n$.

When large datasets are used, the BIC will favour smaller models than the AIC: its penalty $\log(n)$ per parameter exceeds the AIC penalty 2 as soon as $n \geq 8$, and grows with n.

The two criteria have different goals:

	AIC	BIC
Origin	insample error (optimism $2p$)	Bayesian model selection
Penalty per parameter	2	$\log(n)$
Goal	good predictions	identifying the true model
As $n \to \infty$	tends to select too large models	selects the true model, if it is among the candidate models (consistent)

7.7 Training and test sets

Sometimes, when a large (training) dataset is available, one may decide the split the dataset randomly in a

- **training dataset**: data are used for model fitting and for model building or feature selection (this may require e.g. cross validation)
- **test dataset**: this data are used to evaluate the final model (result of model building). An unbiased estimate of the outsample error (i.e. test or generalisation error) based on this test data is

$$\frac{1}{m}\sum_{i=1}^{m}\left(\hat{m}(\mathbf{x}_i) - Y_i\right)^2,$$

  where

  - $(Y_1, \mathbf{x}_1), \ldots, (Y_m, \mathbf{x}_m)$ denote the m observations in the test dataset
  - $\hat{m}$ is estimated from using the training data (this may also be the result from model building, using only the training data).

Note that the training dataset is used for model building or feature selection. This also requires the evaluation of models. For these evaluations the methods from the previous slides can be used (e.g. cross validation, k-fold CV, Mallows' C_p). The test dataset is only used for the evaluation of the final model (estimated and build from using only the training data). The estimate of the outsample error based on the test dataset is the best possible estimate in the

sense that it is unbiased. The observations used for this estimation are independent of the observations in the training data. However, if the number of data points in the test dataset (m) is small, the estimate of the outsample error may show large variance and hence is not reliable.

8 Logistic Regression Analysis for High Dimensional Data

8.1 Breast Cancer Example

- Schmidt *et al.*, 2008, Cancer Research, **68**, 5405-5413
- Gene expression patterns in $n = 200$ breast tumors were investigated ($p = 22283$ genes)
- After surgery the tumors were graded by a pathologist (stage 1,2,3)
- Here the objective is to predict stage 3 from the gene expression data (prediction of binary outcome)
- If the prediction model works well, it can be used to predict the stage from a biopsy sample.

8.2 Data

```
#BiocManager::install("genefu")
#BiocManager::install("breastCancerMAINZ")

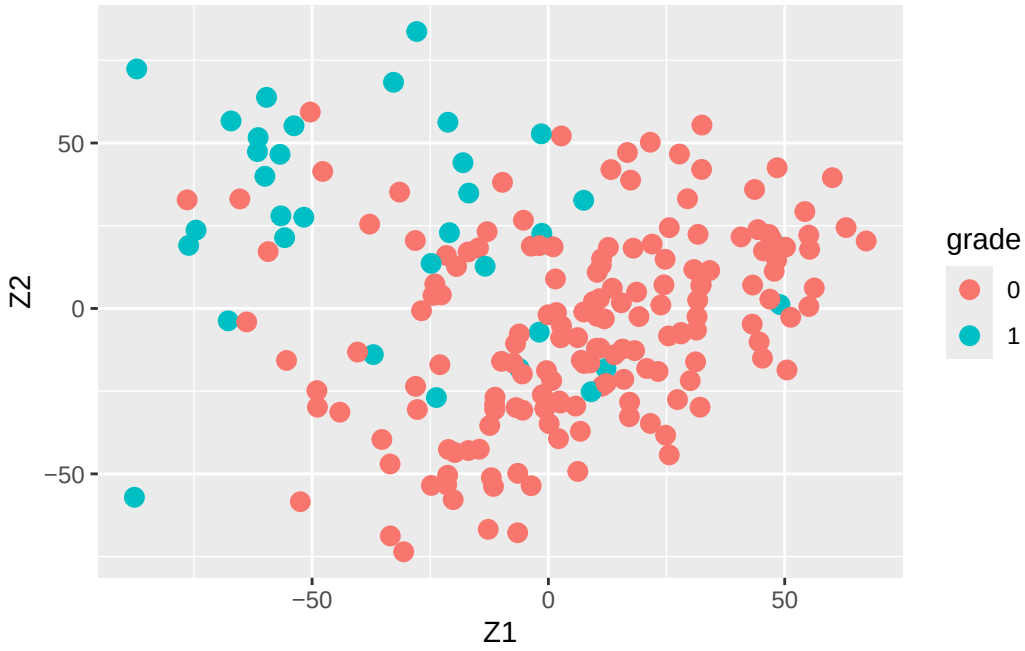
library(genefu)
library(breastCancerMAINZ)
data(mainz)

X <- t(exprs(mainz)) # gene expressions
n <- nrow(X)
H <- diag(n)-1/n*matrix(1,ncol=n,nrow=n)
X <- H%*%X
Y <- ifelse(pData(mainz)$grade==3,1,0)
table(Y)
```

```
#> Y
#>   0   1
#> 165  35
```

```r
svdX <- svd(X)
k <- 2
Zk <- svdX$u[,1:k] %*% diag(svdX$d[1:k])
colnames(Zk) <- paste0("Z",1:k)

Zk |>
  as.data.frame() |>
  mutate(grade = Y |> as.factor()) |>
  ggplot(aes(x= Z1, y = Z2, color = grade)) +
  geom_point(size = 3)
```



8.3 Logistic regression models

Binary outcomes are often analysed with **logistic regression models**.

Let Y denote the binary (1/0, case/control, positive/negative) outcome, and $\mathbf{x}$ the p-dimensional predictor.

Logistic regression assumes

$$Y \mid \mathbf{x} \sim \text{Bernoulli}(\pi(\mathbf{x}))$$

with $\pi(\mathbf{x}) = \mathrm{P}\left[Y = 1 \mid \mathbf{x}\right]$ and

$$\log \frac{\pi(\mathbf{x})}{1 - \pi(\mathbf{x})} = \beta_0 + \beta^T \mathbf{x}.$$

The parameters are typically estimated by maximising the log-likelihood, which is denoted by $l(\ )$, i.e.

$$\widehat{\beta} = \mathrm{ArgMax}_{\beta} l(\beta).$$

- Maximum likelihood is only applicable when $n > p$.
- When $p > n$ penalised maximum likelihood methods are applicable.

8.4 Penalized maximum likelihood

Penalised estimation methods (e.g. lasso and ridge) can als be applied to maximum likelihood, resulting in the **penalised maximum likelihood estimator**.

Lasso:

$$\widehat{\beta} = \mathrm{ArgMax}_{\beta} l(\beta) - \lambda \|\beta\|_1.$$

Ridge:

$$\widehat{\beta} = \mathrm{ArgMax}_{\beta} l(\beta) - \lambda \|\beta\|_2^2.$$

Once the parameters are estimated, the model may be used to compute

$$\widehat{\pi}(\mathbf{x}) = \widehat{\mathrm{P}}\left[Y = 1 \mid \mathbf{x}\right].$$

With these estimated probabilities the prediction rule becomes

$$\begin{aligned} \widehat{\pi}(\mathbf{x}) &\leq c \quad \text{predict } Y = 0 \\ \widehat{\pi}(\mathbf{x}) &> c \quad \text{predict } Y = 1 \end{aligned}$$

with $0 < c < 1$ a threshold that either is fixed (e.g. $c = 1/2$), depends on prior probabilities, or is empirically determined by optimising e.g. the Area Under the ROC Curve (AUC) or by finding a good compromise between sensitivity and specificity.

Note that logistic regression directly models the **Posterior probability** that an observation belongs to class $Y = 1$, given the predictor $\mathbf{x}$.

8.5 Model evaluation

Common model evaluation criteria for binary prediction models are:

- sensitivity = true positive rate (TPR)
- specificity = true negative rate (TNR)
- misclassification error
- area under the ROC curve (AUC)

These criteria can again be estimated via cross validation or via splitting of the data into training and test/validation data.

8.5.1 Sensitivity of a model π with threshold c

Sensitivity is the probability to correctly predict a positive outcome:

$$\text{sens}(\pi, c) = \text{P}_{X^*}\left[\hat{\pi}(\mathbf{X}^*) > c \mid Y^* = 1 \mid \mathcal{T}\right].$$

It is also known as the true positive rate (TPR).

8.5.2 Specificity of a model π with threshold c

Specificity is the probability to correctly predict a negative outcome:

$$\text{spec}(\pi, c) = \text{P}_{X^*}\left[\hat{\pi}(\mathbf{X}^*) \leq c \mid Y^* = 0 \mid \mathcal{T}\right].$$

It is also known as the true negative rate (TNR).

8.5.3 Misclassification error of a model π with threshold c

The misclassification error is the probability to incorrectly predict an outcome:

$$\begin{aligned} \text{mce}(\pi, c) &= \text{P}_{X^*,Y^*}\left[\hat{\pi}(\mathbf{X}) \leq c \text{ and } Y^* = 1 \mid \mathcal{T}\right] \\ &\quad + \text{P}_{X^*,Y^*}\left[\hat{\pi}(\mathbf{X}) > c \text{ and } Y^* = 0 \mid \mathcal{T}\right]. \end{aligned}$$

Note that in the definitions of sensitivity, specificity and the misclassification error, the probabilities refer to the distribution of the $(\mathbf{X}^*, Y^*)$, which is independent of the training data, conditional on the training data. This is in line with the test or generalisation error. The misclassification error is actually the test error when a 0/1 loss function is used. Just as before, the sensitivity, specificity and the misclassification error can also be averaged over the distribution of the training data set, which is in line with the expected test error which has been discussed earlier.

8.5.4 ROC curve of a model π

The Receiver Operating Characteristic (ROC) curve for model π is given by the function

$$\text{ROC} : [0,1] \to [0,1] \times [0,1] : c \mapsto (1 - \text{spec}(\pi, c), \text{sens}(\pi, c)).$$

For when c moves from 1 to 0, the ROC function defines a curve in the plane $[0,1] \times [0,1]$, moving from $(0,0)$ for $c = 1$ to $(1,1)$ for $c = 0$.

The horizontal axis of the ROC curve shows 1-specificity. This is also known as the False Positive Rate (FPR).

8.5.5 Area under the curve (AUC) of a model π

The area under the curve (AUC) for model π is area under the ROC curve and is given by

$$\int_0^1 \text{ROC}(c) dc.$$

Some notes about the AUC:

- AUC=0.5 results when the ROC curve is the diagonal. This corresponds to flipping a coin, i.e. a complete random prediction.
- AUC=1 results from the perfect ROC curve, which is the ROC curve through the points $(0, 0)$, $(0, 1)$ and $(1, 1)$. This ROC curve includes a threshold c such that sensitivity and specificity are equal to one.

8.6 Breast cancer example

8.6.1 Data

```
library(glmnet)

#BiocManager::install("genefu")
#BiocManager::install("breastCancerMAINZ")

library(genefu)
library(breastCancerMAINZ)
data(mainz)

X <- t(exprs(mainz)) # gene expressions
n <- nrow(X)
H <- diag(n)-1/n*matrix(1,ncol=n,nrow=n)
X <- H%*%X
Y <- ifelse(pData(mainz)$grade==3,1,0)
table(Y)
```

```
#> Y
#>   0   1
#> 165  35
```

From the table of the outcomes in Y we read that

- 35 tumors were graded as stage 3 and
- 165 tumors were graded as stage 1 or 2.

In this the stage 3 tumors are referred to as cases or positives and the stage 1 and 2 tumors as controls or negatives.

8.6.2 Training and test dataset

The use of the lasso logistic regression for the prediction of stage 3 breast cancer is illustrated here by

- randomly splitting the dataset into a training dataset (80% of data = 160 tumors) and a test dataset (40 tumors)
- using the training data to select a good λ value in the lasso logistic regression model (through 10-fold CV)
- evaluating the final model by means of the test dataset (ROC Curve, AUC).

```
## Used to provide same results as in previous R version
RNGkind(sample.kind = "Rounding")
set.seed(6977326)
####

n <- nrow(X)
nTrain <- round(0.8*n)
nTrain
```

```
#> [1] 160
```

```
indTrain <- sample(1:n,nTrain)
XTrain <- X[indTrain,]
YTrain <- Y[indTrain]
XTest <- X[-indTrain,]
YTest <- Y[-indTrain]
table(YTest)
```

```
#> YTest
#>  0  1
#> 32  8
```

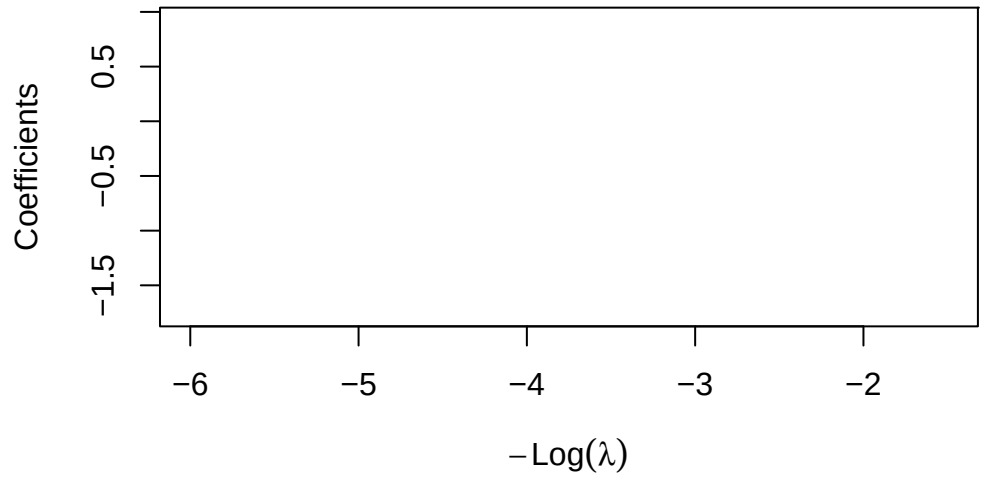
Note that the randomly selected test data has 20% cases of stage 3 tumors. This is a bit higher than the 17.5% in the complete data.

One could also perform the random splitting among the positives and the negatives separately (stratified splitting).

8.6.3 Model fitting based on training data

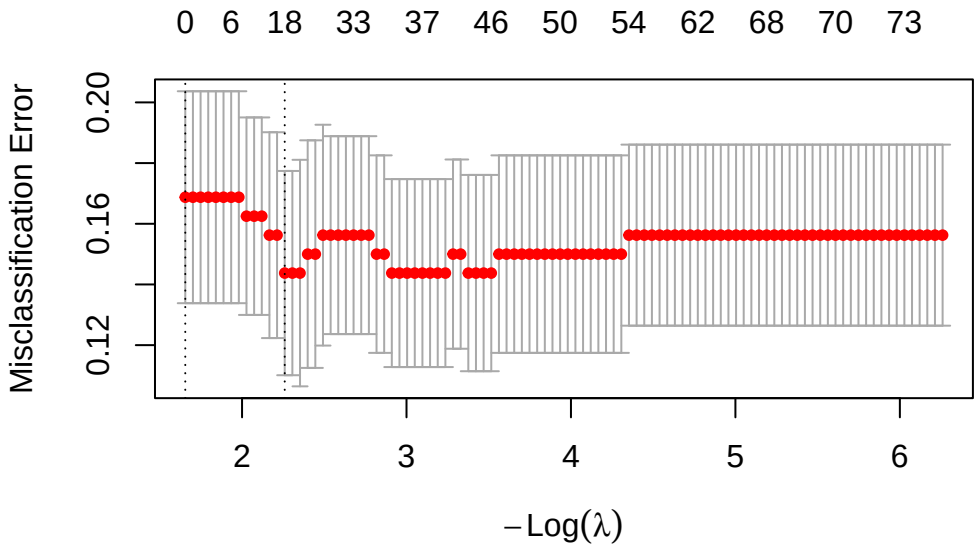
```
mLasso <- glmnet(
  x = XTrain,
  y = YTrain,
  alpha = 1,
  family="binomial")  # lasso: alpha = 1

plot(mLasso, xvar = "lambda", xlim = c(-6,-1.5))
```



```
mCvLasso <- cv.glmnet(
  x = XTrain,
  y = YTrain,
  alpha = 1,
  type.measure = "class",
    family = "binomial")  # lasso alpha = 1

plot(mCvLasso)
```




```
mCvLasso
```

```
#>
#> Call:  cv.glmnet(x = XTrain, y = YTrain, type.measure = "class", alpha = 1,      family =
#>
#> Measure: Misclassification Error
#>
#>     Lambda Index Measure      SE Nonzero
#> min 0.1044    14  0.1437 0.03366      18
#> 1se 0.1911     1  0.1688 0.03492       0
```

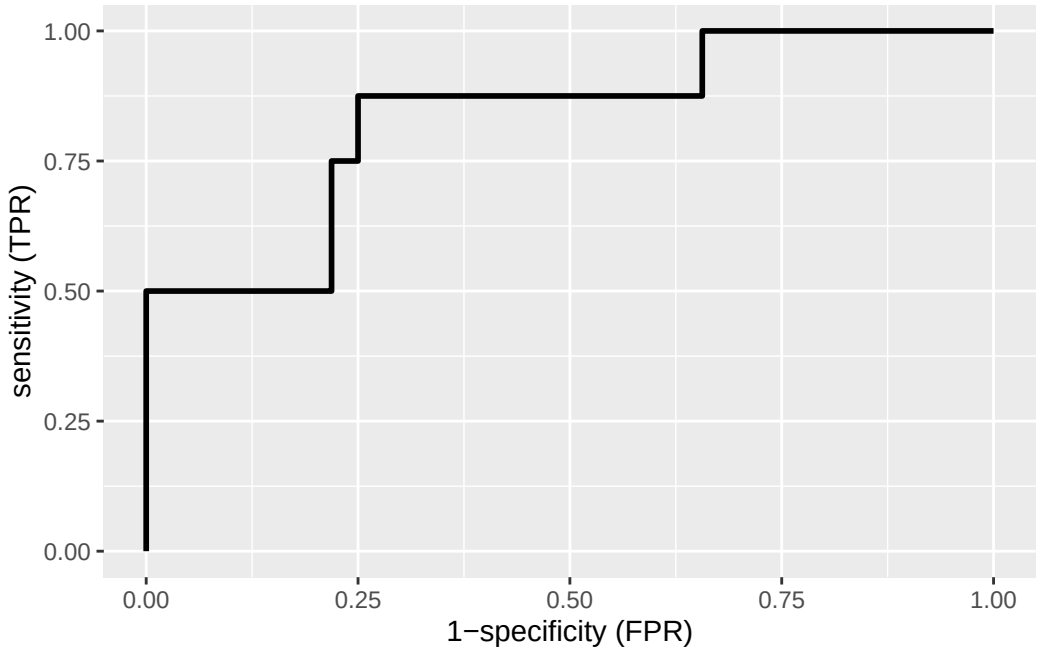
The total misclassification error is used here to select a good value for λ.

```
# BiocManager::install("plotROC")
library(plotROC)

dfLassoOpt <- data.frame(
  pi = predict(mCvLasso,
    newx = XTest,
    s = mCvLasso$lambda.min,
    type = "response") |> c(),
  known.truth = YTest)
```

```
roc <-
  dfLassoOpt  |>
  ggplot(aes(d = known.truth, m = pi)) +
  geom_roc(n.cuts = 0) +
  xlab("1-specificity (FPR)") +
  ylab("sensitivity (TPR)")

roc
```



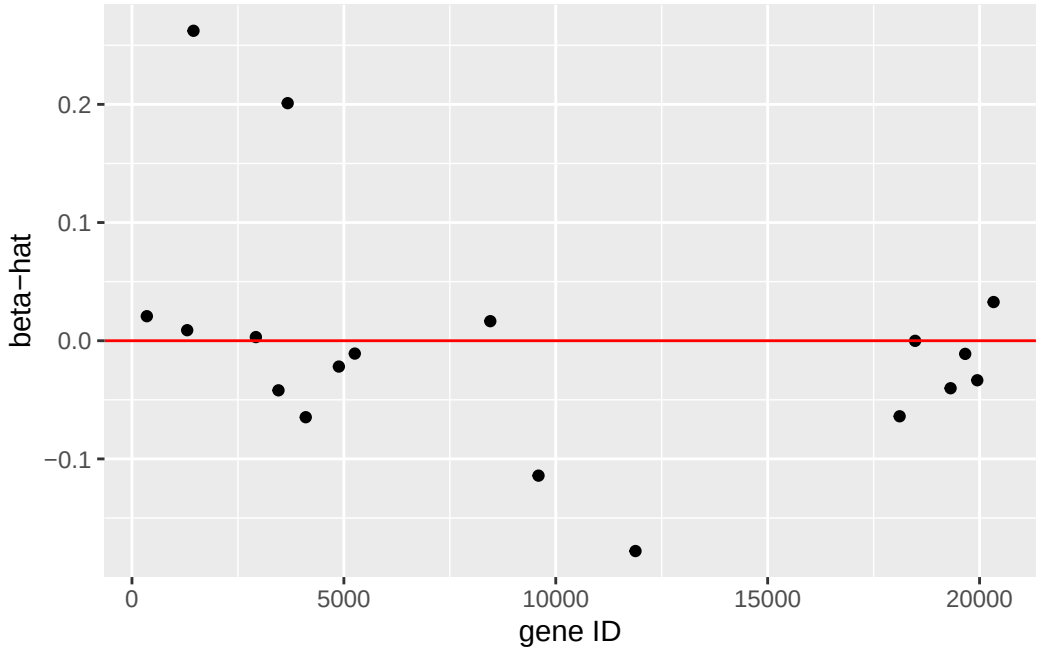

```
calc_auc(roc)
```

```
#>   PANEL COORD group       AUC
#> 1     1     1    -1 0.8320312
```

- The ROC curve is shown for the model based on λ with the smallest misclassification error. The model has an AUC of 0.83.
- Based on this ROC curve an appropriate threshold c can be chosen. For example, from the ROC curve we see that it is possible to attain a specificity and a sensitivity of 75%.
- The sensitivities and specificities in the ROC curve are unbiased (independent test dataset) for the prediction model build from the training data. The estimates of sensitivity and specificity, however, are based on only 40 observations.

```
mLambdaOpt <- glmnet(x = XTrain,
  y = YTrain,
  alpha = 1,
  lambda = mCvLasso$lambda.min,
  family="binomial")

qplot(
  summary(coef(mLambdaOpt))[-1,1],
  summary(coef(mLambdaOpt))[-1,3]) +
  xlab("gene ID") +
  ylab("beta-hat") +
  geom_hline(yintercept = 0, color = "red")
```

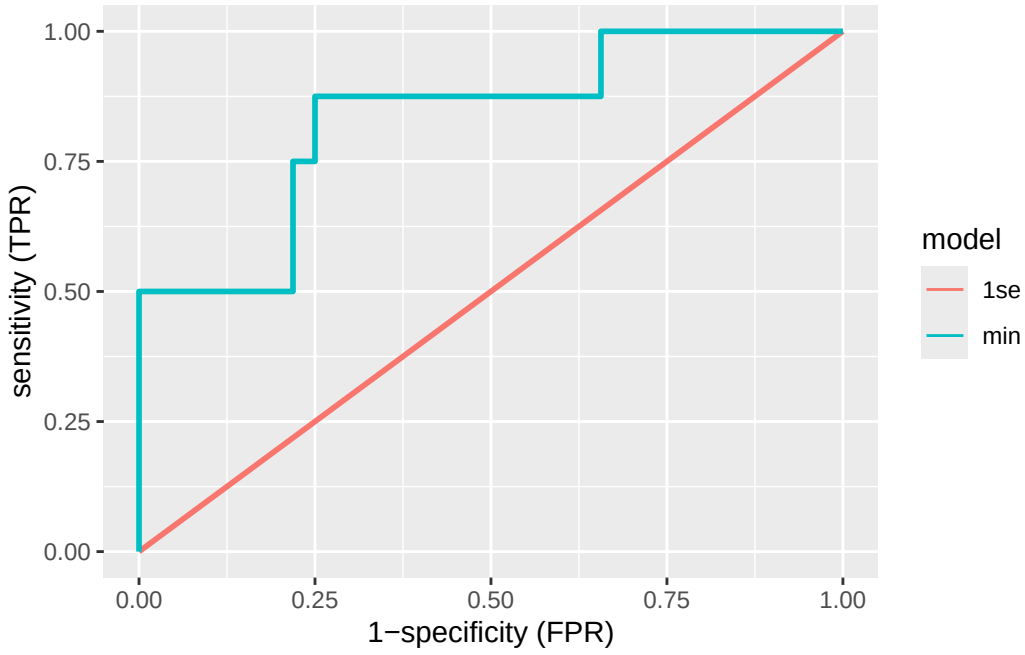


- The model with the optimal λ has only 19 non-zero parameter estimates.
- Thus only 19 genes are involved in the prediction model.
- These 19 parameter estimates are plotting in the graph. A listing of the model output would show the names of the genes.

```r
dfLasso1se <- data.frame(
  pi = predict(mCvLasso,
    newx = XTest,
    s = mCvLasso$lambda.1se,
    type = "response") |> c(),
  known.truth = YTest)

roc <-
  rbind(
    dfLassoOpt |>
      mutate(model = "min"),
    dfLasso1se |>
      mutate(model = "1se")
  ) |>
  ggplot(aes(d = known.truth, m = pi, color = model)) +
  geom_roc(n.cuts = 0) +
  xlab("1-specificity (FPR)") +
  ylab("sensitivity (TPR)")

roc
```



```
calc_auc(roc)
```

```
#>   PANEL COORD group model       AUC
#> 1     1     1     1   1se 0.5000000
#> 2     1     1     2   min 0.8320312
```

- When using the λ of the optimal model up to 1 standard deviation, a diagonal ROC curve is obtained and hence AUC is 0.5.
- This prediction model is thus equivalent to flipping a coin for making the prediction.
- The reason is that with this choice of λ (strong penalisation) almost all predictors are removed from the model.
- Therefore, do never blindly choose for the "optimal'' λ as defined here, but assess the performance of the model first.

```
mLambda1se <- glmnet(x = XTrain,
  y = YTrain,
  alpha = 1,
  lambda = mCvLasso$lambda.1se,
  family="binomial")

mLambda1se |>
  coef() |>
  summary()
```

```
#> 22284 x 1 sparse Matrix of class "dgCMatrix", with 2 entries
#>   i j         x
#> 1 1 1 -1.594512
#> 2 2 1  0.000000
```

8.7 The Elastic Net

The lasso and ridge regression have positive and negative properties.

- Lasso
  - positive: sparse solution
  - negative: at most $\min(n, p)$ predictors can be selected

  - negative: tend to select one predictor among a group of highly correlated predictors

- Ridge
  - negative: no sparse solution
  - positive: more than $\min(n, p)$ predictors can be selected

A compromise between lasso and ridge: the **elastic net**:

$$\hat{\beta} = \text{ArgMax}_{\beta} l(\beta) - \gamma_1 \|\beta\|_1 - \gamma_2 \|\beta\|_2^2.$$

The elastic gives a sparse solution with potentially more than $\min(n, p)$ predictors.

The `glmnet` R function uses the following parameterisation,

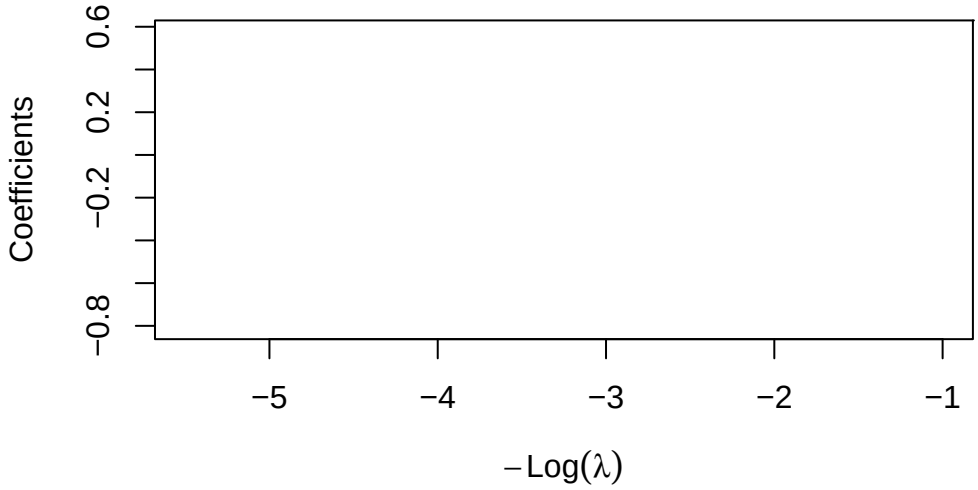
$$\hat{\beta} = \text{ArgMax}_{\beta} l(\beta) - \lambda\alpha \|\beta\|_1 - \lambda(1-\alpha) \|\beta\|_2^2.$$

- α parameter gives weight to L_1 penalty term (hence $\alpha = 1$ gives the lasso, and $\alpha = 0$ gives ridge).
- a λ parameter to give weight to the penalisation
- Note that the combination of λ and α gives the same flexibility as the combination of the parameters λ_1 and λ_2.

8.7.1 Breast cancer example

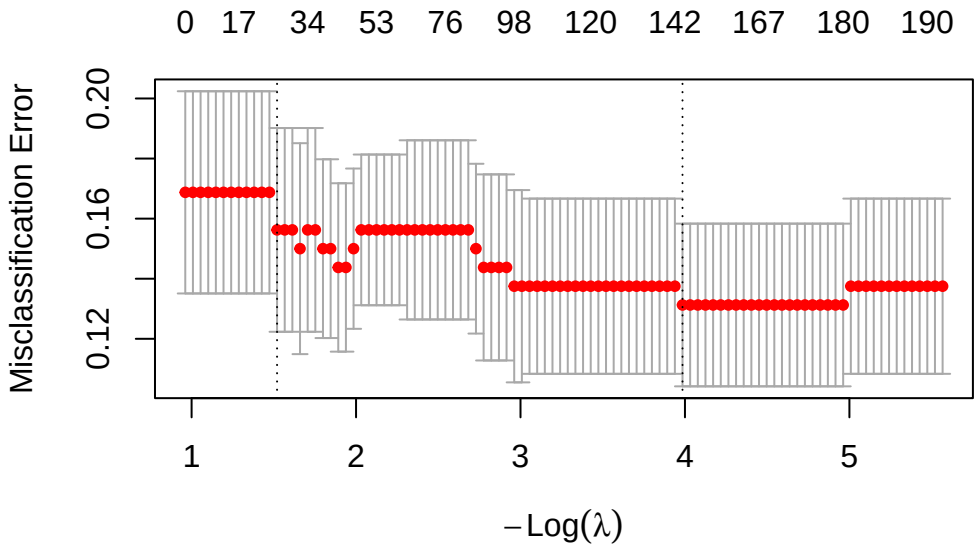
```
mElastic <- glmnet(
  x = XTrain,
  y = YTrain,
  alpha = 0.5,
  family="binomial")  # elastic net

plot(mElastic, xvar = "lambda",xlim=c(-5.5,-1))
```




```
mCvElastic <- cv.glmnet(x = XTrain,
  y = YTrain,
  alpha = 0.5,
  family = "binomial",
    type.measure = "class")  # elastic net

plot(mCvElastic)
```




```
mCvElastic
```

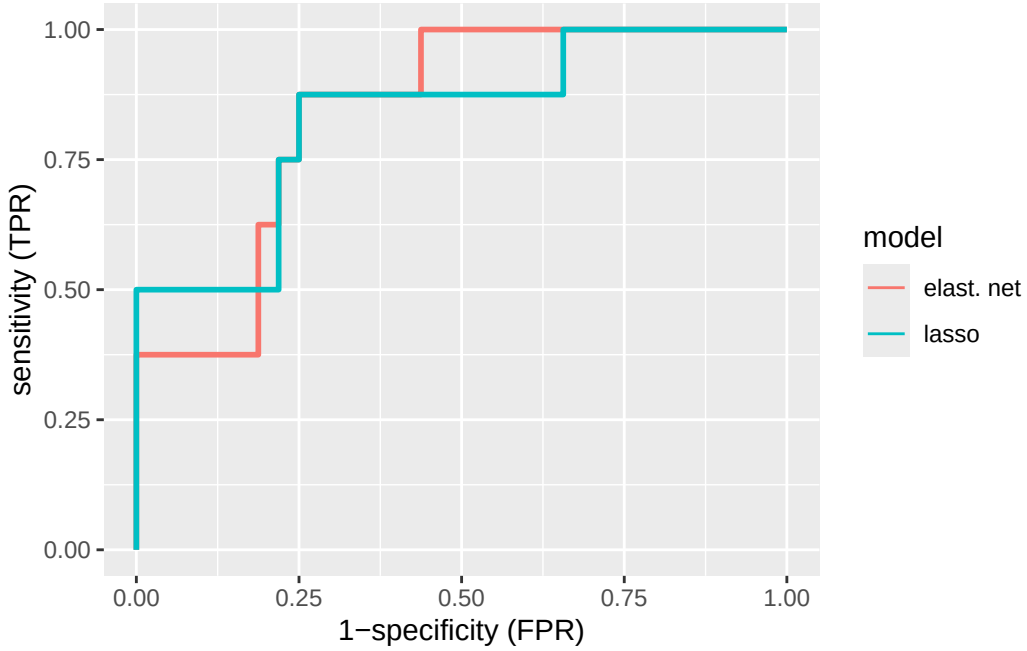
```
#>
#> Call:  cv.glmnet(x = XTrain, y = YTrain, type.measure = "class", alpha = 0.5,      family
#>
#> Measure: Misclassification Error
#>
#>      Lambda Index Measure      SE Nonzero
#> min 0.01859    66  0.1313 0.02708     148
#> 1se 0.21876    13  0.1562 0.03391      26
```

```
dfElast <- data.frame(
  pi = predict(mElastic,
    newx = XTest,
    s = mCvElastic$lambda.min,
    type = "response") |> c(),
  known.truth = YTest)

roc <- rbind(
  dfLassoOpt |> mutate(model = "lasso"),
  dfElast |> mutate(model = "elast. net")) |>
  ggplot(aes(d = known.truth, m = pi, color = model)) +
```

```
  geom_roc(n.cuts = 0) +
  xlab("1-specificity (FPR)") +
  ylab("sensitivity (TPR)")

roc
```




```
calc_auc(roc)
```

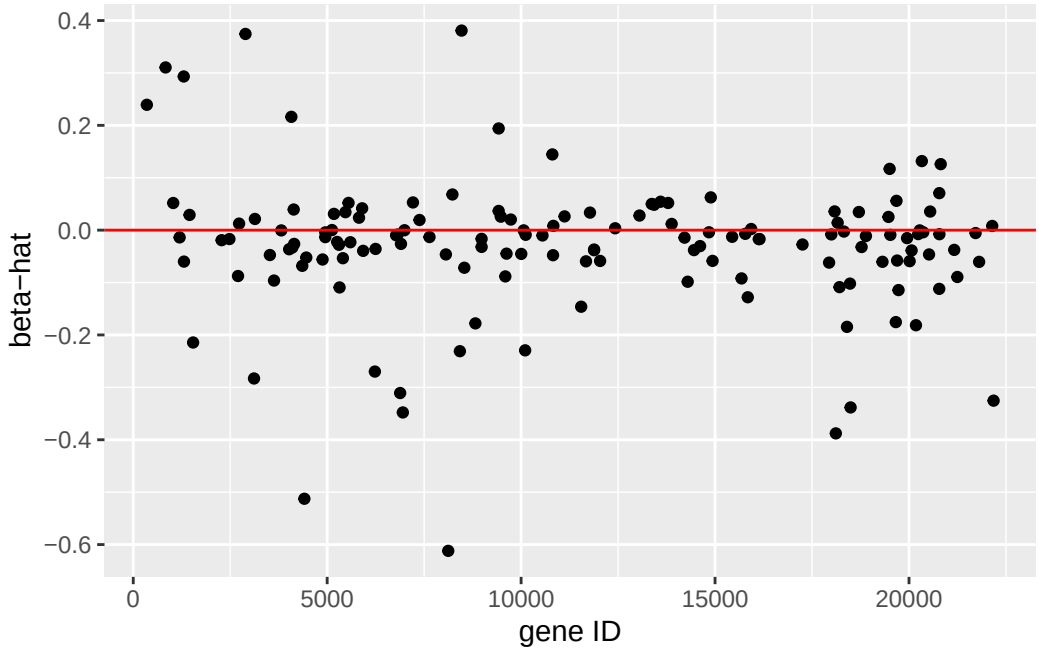
```
#>   PANEL COORD group      model       AUC
#> 1     1     1     1 elast. net 0.8398438
#> 2     1     1     2      lasso 0.8320312
```

- More parameters are used than for the lasso, but the performance does not improve.

```
mElasticOpt <- glmnet(x = XTrain,
  y = YTrain,
  alpha = 0.5,
  lambda = mCvElastic$lambda.min,
  family="binomial")

qplot(
  summary(coef(mElasticOpt))[-1,1],
```

```
  summary(coef(mElasticOpt))[-1,3]) +
  xlab("gene ID") +
  ylab("beta-hat") +
  geom_hline(yintercept = 0, color = "red")
```



Acknowledgement

- Olivier Thas for sharing his materials of Analysis of High Dimensional Data 2019-2020, which I used as the starting point for this chapter.

Session info

Session info

```
#> [1] "2026-10-19 21:26:02 CEST"
```

```
#> - Session info ---------------------------------------------------------------
#>  setting  value
#>  version  R version 4.6.1 (2026-06-24)
```

```
#>  os       macOS Tahoe 26.1
#>  system   aarch64, darwin23
#>  ui       X11
#>  language (EN)
#>  collate  nl_BE.UTF-8
#>  ctype    nl_BE.UTF-8
#>  tz       Europe/Brussels
#>  date     2026-10-19
#>  pandoc   3.10.2 @ /opt/homebrew/bin/ (via rmarkdown)
#>  quarto   1.9.37 @ /usr/local/bin/quarto
#>
#> - Packages -------------------------------------------------------------------
#>  package           * version    date (UTC) lib source
#>  AIMS              * 1.44.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  AnnotationDbi       1.74.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  Biobase           * 2.72.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  BiocFileCache       3.2.0      2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  BiocGenerics      * 0.58.1     2026-05-14 [1] https://bioc-release.r-universe.dev (R 4.6
#>  biomaRt           * 2.68.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  Biostrings          2.80.1     2026-05-22 [1] https://bioc-release.r-universe.dev (R 4.6
#>  bit                 4.6.0      2025-03-06 [1] CRAN (R 4.6.0)
#>  bit64               4.8.2      2026-05-19 [1] CRAN (R 4.6.0)
#>  blob                1.3.0      2026-01-14 [1] CRAN (R 4.6.0)
#>  bootstrap           2019.6     2019-06-17 [1] CRAN (R 4.6.0)
#>  breastCancerMAINZ * 1.50.0     2026-05-05 [1] Bioconductor 3.23 (R 4.6.1)
#>  cachem              1.1.0      2024-05-16 [1] CRAN (R 4.6.0)
#>  class               7.3-23     2025-01-01 [1] CRAN (R 4.6.1)
#>  cli                 3.6.6      2026-04-09 [1] CRAN (R 4.6.0)
#>  cluster             2.1.8.2    2026-02-05 [1] CRAN (R 4.6.1)
#>  codetools           0.2-20     2024-03-31 [1] CRAN (R 4.6.1)
#>  crayon              1.5.3      2024-06-20 [1] CRAN (R 4.6.0)
#>  curl                7.1.0      2026-04-22 [1] CRAN (R 4.6.0)
#>  DAAG              * 1.25.7     2025-10-09 [1] CRAN (R 4.6.0)
#>  data.table          1.18.4     2026-05-06 [1] CRAN (R 4.6.0)
#>  DBI                 1.3.0      2026-02-25 [1] CRAN (R 4.6.0)
#>  dbplyr              2.6.0      2026-06-17 [1] CRAN (R 4.6.0)
#>  deldir              2.0-4      2024-02-28 [1] CRAN (R 4.6.0)
#>  digest              0.6.39     2025-11-19 [1] CRAN (R 4.6.0)
#>  dplyr             * 1.2.1      2026-04-03 [1] CRAN (R 4.6.0)
#>  e1071             * 1.7-17     2025-12-18 [1] CRAN (R 4.6.0)
#>  evaluate            1.0.5      2025-08-27 [1] CRAN (R 4.6.0)
#>  farver              2.1.2      2024-05-13 [1] CRAN (R 4.6.0)
#>  fastmap             1.2.0      2024-05-15 [1] CRAN (R 4.6.0)
```

```
#>  filelock            1.0.3     2023-12-11 [1] CRAN (R 4.6.0)
#>  forcats           * 1.0.1     2025-09-25 [1] CRAN (R 4.6.0)
#>  foreach             1.5.2     2022-02-02 [1] CRAN (R 4.6.0)
#>  future              1.75.0    2026-07-20 [1] CRAN (R 4.6.1)
#>  future.apply        1.20.2    2026-02-20 [1] CRAN (R 4.6.0)
#>  genefu            * 2.44.0    2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  generics          * 0.1.4     2025-05-09 [1] CRAN (R 4.6.0)
#>  ggforce           * 0.5.0     2025-06-18 [1] CRAN (R 4.6.0)
#>  ggplot2           * 4.0.3     2026-04-22 [1] CRAN (R 4.6.0)
#>  glmnet            * 5.0       2026-05-04 [1] CRAN (R 4.6.0)
#>  globals             0.19.1    2026-03-13 [1] CRAN (R 4.6.0)
#>  glue                1.8.1     2026-04-17 [1] CRAN (R 4.6.0)
#>  gridExtra         * 2.3.1     2026-06-25 [1] CRAN (R 4.6.1)
#>  gtable              0.3.6     2024-10-25 [1] CRAN (R 4.6.0)
#>  hms                 1.1.4     2025-10-17 [1] CRAN (R 4.6.0)
#>  htmltools           0.5.9     2025-12-04 [1] CRAN (R 4.6.0)
#>  httr                1.4.8     2026-02-13 [1] CRAN (R 4.6.0)
#>  httr2               1.2.3     2026-06-23 [1] CRAN (R 4.6.1)
#>  iC10              * 2.0.3     2026-08-04 [1] CRAN (R 4.6.1)
#>  iC10TrainingData    2.0.1     2024-07-16 [1] CRAN (R 4.6.0)
#>  impute              1.86.0    2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  interp              1.1-6     2024-01-26 [1] CRAN (R 4.6.0)
#>  IRanges             2.46.0    2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  iterators           1.0.14    2022-02-05 [1] CRAN (R 4.6.0)
#>  jpeg                0.1-11    2025-03-21 [1] CRAN (R 4.6.0)
#>  jsonlite            2.0.0     2025-03-27 [1] CRAN (R 4.6.0)
#>  KEGGREST            1.52.0    2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  KernSmooth          2.23-26   2025-01-01 [1] CRAN (R 4.6.1)
#>  knitr               1.51      2025-12-20 [1] CRAN (R 4.6.0)
#>  labeling            0.4.3     2023-08-29 [1] CRAN (R 4.6.0)
#>  latex2exp         * 0.9.8     2026-01-09 [1] CRAN (R 4.6.0)
#>  lattice             0.22-9    2026-02-09 [1] CRAN (R 4.6.1)
#>  latticeExtra        0.6-31    2025-09-10 [1] CRAN (R 4.6.0)
#>  lava                1.9.3     2026-08-20 [1] CRAN (R 4.6.1)
#>  lifecycle           1.0.5     2026-01-08 [1] CRAN (R 4.6.0)
#>  limma               3.68.4    2026-05-31 [1] https://bioc-release.r-universe.dev (R 4.6
#>  listenv             1.0.0     2026-06-22 [1] CRAN (R 4.6.1)
#>  lubridate         * 1.9.5     2026-02-04 [1] CRAN (R 4.6.0)
#>  magrittr            2.0.5     2026-04-04 [1] CRAN (R 4.6.0)
#>  MASS                7.3-65    2025-02-28 [1] CRAN (R 4.6.1)
#>  Matrix            * 1.7-5     2026-03-21 [1] CRAN (R 4.6.1)
#>  mclust              6.1.3     2026-07-05 [1] CRAN (R 4.6.1)
#>  memoise             2.0.1     2021-11-26 [1] CRAN (R 4.6.0)
```

```
#>  mgcv             * 1.9-4      2025-11-07 [1] CRAN (R 4.6.1)
#>  nlme             * 3.1-169    2026-03-27 [1] CRAN (R 4.6.1)
#>  otel               0.2.0      2025-08-29 [1] CRAN (R 4.6.0)
#>  pamr               1.57       2024-07-01 [1] CRAN (R 4.6.0)
#>  parallelly         1.48.0     2026-06-29 [1] CRAN (R 4.6.1)
#>  pillar             1.11.1     2025-09-17 [1] CRAN (R 4.6.0)
#>  pkgconfig          2.0.3      2019-09-22 [1] CRAN (R 4.6.0)
#>  plotROC          * 2.3.3      2025-08-25 [1] CRAN (R 4.6.0)
#>  plyr               1.8.9      2023-10-02 [1] CRAN (R 4.6.0)
#>  png                0.1-9      2026-03-15 [1] CRAN (R 4.6.0)
#>  polyclip           1.10-7     2024-07-23 [1] CRAN (R 4.6.0)
#>  prettyunits        1.2.0      2023-09-24 [1] CRAN (R 4.6.0)
#>  prodlim          * 2026.03.11 2026-03-11 [1] CRAN (R 4.6.0)
#>  progress           1.2.3      2023-12-06 [1] CRAN (R 4.6.0)
#>  proxy              0.4-29     2025-12-29 [1] CRAN (R 4.6.0)
#>  purrr            * 1.2.2      2026-04-10 [1] CRAN (R 4.6.0)
#>  R6                 2.6.1      2025-02-15 [1] CRAN (R 4.6.0)
#>  rappdirs           0.3.4      2026-01-17 [1] CRAN (R 4.6.0)
#>  rbibutils          2.4.1      2026-01-21 [1] CRAN (R 4.6.0)
#>  RColorBrewer       1.1-3      2022-04-03 [1] CRAN (R 4.6.0)
#>  Rcpp               1.1.1-1.1  2026-04-24 [1] CRAN (R 4.6.0)
#>  Rdpack             2.6.6      2026-02-08 [1] CRAN (R 4.6.0)
#>  readr            * 2.2.0      2026-02-19 [1] CRAN (R 4.6.0)
#>  rlang              1.2.0      2026-04-06 [1] CRAN (R 4.6.0)
#>  rmarkdown          2.31       2026-03-26 [1] CRAN (R 4.6.0)
#>  rmeta              3.0        2018-03-20 [1] CRAN (R 4.6.0)
#>  RSQLite            3.53.2     2026-06-17 [1] CRAN (R 4.6.0)
#>  S4Vectors          0.50.1     2026-05-03 [1] https://bioc-release.r-universe.dev (R 4.6
#>  S7                 0.2.2      2026-04-22 [1] CRAN (R 4.6.0)
#>  scales             1.4.0      2025-04-24 [1] CRAN (R 4.6.0)
#>  SemiPar          * 1.0-4.2    2018-04-16 [1] CRAN (R 4.6.1)
#>  Seqinfo            1.2.0      2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  sessioninfo        1.2.4      2026-06-04 [1] CRAN (R 4.6.0)
#>  shape              1.4.6.1    2024-02-23 [1] CRAN (R 4.6.0)
#>  statmod            1.5.2      2026-05-17 [1] CRAN (R 4.6.0)
#>  stringi            1.8.7      2025-03-27 [1] CRAN (R 4.6.0)
#>  stringr          * 1.6.0      2025-11-04 [1] CRAN (R 4.6.0)
#>  SuppDists          1.1-9.9    2025-03-24 [1] CRAN (R 4.6.0)
#>  survcomp         * 1.62.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  survival         * 3.8-6      2026-01-16 [1] CRAN (R 4.6.1)
#>  survivalROC        1.0.3.1    2022-12-05 [1] CRAN (R 4.6.0)
#>  tibble           * 3.3.1      2026-01-11 [1] CRAN (R 4.6.0)
#>  tidyr            * 1.3.2      2025-12-19 [1] CRAN (R 4.6.0)
```

```
#>  tidyselect          1.2.1      2024-03-11 [1] CRAN (R 4.6.0)
#>  tidyverse         * 2.0.0      2023-02-22 [1] CRAN (R 4.6.0)
#>  timechange          0.4.0      2026-01-29 [1] CRAN (R 4.6.0)
#>  tweenr              2.0.3      2024-02-26 [1] CRAN (R 4.6.0)
#>  tzdb                0.5.0      2025-03-15 [1] CRAN (R 4.6.0)
#>  utf8                1.2.6      2025-06-08 [1] CRAN (R 4.6.0)
#>  vctrs               0.7.3      2026-04-11 [1] CRAN (R 4.6.0)
#>  vroom               1.7.1      2026-03-31 [1] CRAN (R 4.6.0)
#>  withr               3.0.3      2026-06-19 [1] CRAN (R 4.6.0)
#>  xfun                0.59       2026-06-19 [1] CRAN (R 4.6.0)
#>  XVector             0.52.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  yaml                2.3.12     2025-12-10 [1] CRAN (R 4.6.0)
#>
#>  [1] /Library/Frameworks/R.framework/Versions/4.6/Resources/library
#>  * -- Packages attached to the search path.
#>
#> ------------------------------------------------------------------------------
```